



**Universitatea  
Transilvania  
din Brașov**  
FACULTATEA DE MATEMATICĂ  
ȘI INFORMATICĂ

Programul de studii:  
Informatică

## Lucrare de licență / disertație

**Favigon - Platformă web pentru proiectarea vizuală a interfețelor web, asistată de inteligență artificială, partajarea și conversia lor în cod**

**Autor:** Vasile Rareș Mihail  
**Coordonator științific:** Lect. dr. Spridon Delia-Elena

Brașov, 2026



# Rezumat

Scopul lucrării este proiectarea și implementarea platformei *Favigon*, o aplicație web full-stack construită pentru a reduce distanța dintre ideea de interfață, editarea vizuală și codul final. Problema de la care a pornit proiectul este una practică: în multe fluxuri de lucru actuale, designul este realizat într-un instrument separat, implementarea se face manual, iar trecerea dintre cele două etape consumă timp și introduce neclarități.

Soluția propusă combină într-un singur produs un editor vizual de tip canvas, generare asistată de inteligență artificială, export în cod și o zonă socială pentru publicarea și reutilizarea proiectelor. Din punct de vedere tehnic, aplicația folosește Angular 20 pe frontend, ASP.NET Core 9 pe backend, PostgreSQL pentru persistență și Docker Compose pentru orchestrarea serviciilor principale. Pentru expunerea controlată în exterior este utilizat și Cloudflare Tunnel.

Componenta centrală a aplicației este editorul canvas. Acesta suportă lucru pe mai multe pagini, selecție, gesturi de manipulare, istoric, persistare automată și previzualizare. Pentru a evita legarea directă a editorului de codul generat, proiectul introduce o reprezentare intermediară a interfeței, reutilizată atât în editor, cât și în motorul de conversie. Această alegere a făcut posibilă separarea clară dintre editare, validare și export.

Partea de AI nu produce direct cod final. În schimb, a fost implementat un pipeline în trei faze: extragerea intenției, generarea structurii și stilizarea. Rezultatul este validat, poate fi reparat atunci când nu respectă schema așteptată și este apoi transformat în HTML, React sau Angular de motorul de conversie. Separarea pe etape oferă control mai bun asupra ieșirii și ajută la localizarea mai rapidă a problemelor.

Lucrarea tratează și componentele necesare pentru ca aplicația să fie utilizabilă ca produs: autentificare cu JWT în cookie-uri, OAuth, resetare parolă, autentificare în doi pași, proiecte publice și private, funcții de tip like, star, follow și fork, precum și măsuri de securitate aplicate în backend. La nivel de validare, backend-ul și convertorul sunt acoperite de 108 teste automate, iar fluxurile principale au fost verificate și prin scenarii funcționale.

Contribuția principală a lucrării constă în integrarea coerentă a editorului canvas, a reprezentării intermediare, a pipeline-ului AI și a exportului multi-framework într-o singură aplicație. Rezultatul nu este prezentat ca înlocuitor al unor produse comerciale mature, ci ca o platformă software funcțională care demonstrează fezabilitatea unui flux integrat de tip design-to-code, explicabil și controlabil tehnic.

# Cuprins

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Rezumat</b>                                                | <b>i</b>  |
| <b>1 Introducere</b>                                          | <b>1</b>  |
| 1.1 Contextul actual al temei                                 | 1         |
| 1.2 Problema pe care o urmărește lucrarea                     | 1         |
| 1.3 Motivația alegerii temei                                  | 1         |
| 1.4 Obiectivele proiectului                                   | 2         |
| 1.5 Contribuțiile proprii                                     | 2         |
| 1.6 Metoda de lucru                                           | 3         |
| 1.7 Structura lucrării                                        | 3         |
| <b>2 Analiza domeniului și studiu comparativ</b>              | <b>4</b>  |
| 2.1 Evoluția spațiului design-to-code                         | 4         |
| 2.2 Criterii de comparație                                    | 4         |
| 2.3 Aplicații relevante din aceeași zonă                      | 5         |
| 2.3.1 Figma                                                   | 5         |
| 2.3.2 Webflow                                                 | 5         |
| 2.3.3 Framer                                                  | 5         |
| 2.3.4 Builder.io                                              | 5         |
| 2.4 Tabel comparativ                                          | 6         |
| 2.5 Poziționarea proiectului Favigon                          | 6         |
| 2.6 Golul pe care îl acoperă Favigon                          | 7         |
| 2.7 Concluzii intermediare                                    | 7         |
| <b>3 Fundamente teoretice și tehnologii utilizate</b>         | <b>8</b>  |
| 3.1 Considerații generale                                     | 8         |
| 3.2 Aplicații single-page și rolul frontend-ului modern       | 8         |
| 3.3 Arhitectura client-server și API-ul REST                  | 8         |
| 3.4 Autentificare, token-uri și securitate la nivel de cerere | 9         |
| 3.5 Reprezentarea intermediară a interfeței                   | 9         |
| 3.6 JSON Schema și ieșiri structurate pentru AI               | 10        |
| 3.7 Baza de date relațională și persistența                   | 10        |
| 3.8 Orchestrare și infrastructură                             | 11        |
| 3.9 De ce această combinație tehnologică a fost potrivită     | 12        |
| 3.10 Concluzii intermediare                                   | 12        |
| <b>4 Analiza și proiectarea sistemului</b>                    | <b>13</b> |
| 4.1 Pornirea de la cerințe                                    | 13        |
| 4.2 Cerințe funcționale                                       | 13        |
| 4.3 Cerințe nefuncționale                                     | 13        |
| 4.4 Actorii principali                                        | 14        |
| 4.5 Scenarii principale de utilizare                          | 15        |
| 4.5.1 Scenariul 1: creare și editare proiect                  | 15        |
| 4.5.2 Scenariul 2: generare asistată de AI                    | 15        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 4.5.3    | Scenariul 3: export în cod                                          | 15        |
| 4.5.4    | Scenariul 4: distribuire și reutilizare                             | 16        |
| 4.6      | Arhitectura generală                                                | 16        |
| 4.7      | Organizarea backend-ului pe straturi                                | 16        |
| 4.8      | Organizarea frontend-ului pe feature-uri                            | 17        |
| 4.9      | Modelul de date conceptual                                          | 18        |
| 4.10     | Fluxurile principale                                                | 19        |
| 4.10.1   | Fluxul de autentificare                                             | 19        |
| 4.10.2   | Fluxul de editare                                                   | 19        |
| 4.10.3   | Fluxul de export                                                    | 19        |
| 4.10.4   | Fluxul AI                                                           | 19        |
| 4.11     | Decizii de proiectare care au influențat cel mai mult implementarea | 19        |
| 4.12     | Concluzii intermediare                                              | 20        |
| <b>5</b> | <b>Implementarea platformei Favigon</b>                             | <b>21</b> |
| 5.1      | Privire de ansamblu asupra implementării                            | 21        |
| 5.2      | Rutare și pornirea aplicației frontend                              | 21        |
| 5.3      | Modulul de autentificare și profil                                  | 22        |
| 5.3.1    | Fluxurile de bază                                                   | 22        |
| 5.3.2    | JWT în cookie și refresh token                                      | 22        |
| 5.3.3    | Autentificarea în doi pași                                          | 23        |
| 5.3.4    | Profilul utilizatorului                                             | 23        |
| 5.4      | Gestionarea proiectelor                                             | 23        |
| 5.4.1    | Ciclul de viață al proiectelor                                      | 23        |
| 5.4.2    | Persistarea designului                                              | 23        |
| 5.4.3    | Thumbnail și asset-uri                                              | 24        |
| 5.5      | Funcționalități sociale și explorare                                | 24        |
| 5.6      | Suprafața API și distribuția responsabilităților                    | 25        |
| 5.7      | Editorul canvas                                                     | 26        |
| 5.7.1    | Rolul editorului în întregul produs                                 | 26        |
| 5.7.2    | Starea editorului                                                   | 27        |
| 5.7.3    | Viewport, selecție și gesturi                                       | 27        |
| 5.7.4    | Istoric și clipboard                                                | 27        |
| 5.7.5    | Managementul paginilor                                              | 28        |
| 5.7.6    | Preview-ul interactiv                                               | 28        |
| 5.7.7    | Persistare și integrare cu restul aplicației                        | 28        |
| 5.7.8    | Persistența locală a conversației AI                                | 29        |
| 5.8      | Decizii de implementare cu impact major                             | 29        |
| 5.9      | Concluzii intermediare                                              | 30        |
| <b>6</b> | <b>Generarea asistată de AI și conversia interfețelor în cod</b>    | <b>31</b> |
| 6.1      | De ce nu este suficientă generarea directă                          | 31        |
| 6.2      | Principiul pipeline-ului în trei faze                               | 31        |
| 6.3      | Faza 1: extragerea intenției                                        | 32        |
| 6.4      | Faza 2: generarea structurii                                        | 32        |
| 6.5      | Faza 3: stilizarea                                                  | 33        |
| 6.6      | Integrarea cu OpenAI API                                            | 33        |
| 6.7      | Structura cererilor AI și feedback incremental                      | 33        |

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| 6.8      | Rolul ieșirilor structurate . . . . .                                       | 35        |
| 6.9      | Mecanisme de validare și reparare . . . . .                                 | 35        |
| 6.10     | Reprezentarea intermediară în detaliu . . . . .                             | 36        |
| 6.11     | Maparea dintre canvas și IR . . . . .                                       | 36        |
| 6.12     | Exemplu concret de flux de la prompt la IR și export . . . . .              | 36        |
| 6.13     | Motorul de conversie . . . . .                                              | 37        |
| 6.13.1   | Separarea în proiect dedicat . . . . .                                      | 37        |
| 6.13.2   | Validarea . . . . .                                                         | 37        |
| 6.13.3   | Generarea single-page și multi-page . . . . .                               | 37        |
| 6.13.4   | Responsive output . . . . .                                                 | 37        |
| 6.14     | Legătura dintre AI și convertor . . . . .                                   | 38        |
| 6.15     | Limitări actuale . . . . .                                                  | 38        |
| 6.16     | Concluzii intermediare . . . . .                                            | 39        |
| <b>7</b> | <b>Scenarii de utilizare și evaluare practică</b>                           | <b>40</b> |
| 7.1      | Rolul acestui capitol în contextul lucrării . . . . .                       | 40        |
| 7.2      | Criteriile urmărite în evaluarea practică . . . . .                         | 40        |
| 7.3      | Scenariul 1: creare manuală și export al unui proiect . . . . .             | 40        |
| 7.4      | Scenariul 2: generare pornind de la prompt și rafinare ulterioară . . . . . | 42        |
| 7.5      | Scenariul 3: publicare, explorare și reutilizare prin fork . . . . .        | 43        |
| 7.6      | Observații asupra comportamentului sistemului . . . . .                     | 43        |
| 7.7      | Concluzii intermediare . . . . .                                            | 44        |
| <b>8</b> | <b>Securitate, testare și validare</b>                                      | <b>45</b> |
| 8.1      | Rolul acestei etape în proiect . . . . .                                    | 45        |
| 8.2      | Măsurile de securitate implementate . . . . .                               | 45        |
| 8.2.1    | Autentificare și sesiune . . . . .                                          | 45        |
| 8.2.2    | Limitarea expunerii API-ului . . . . .                                      | 45        |
| 8.2.3    | Middleware și antete de securitate . . . . .                                | 45        |
| 8.2.4    | Upload-uri și asset-uri . . . . .                                           | 46        |
| 8.3      | Strategia de testare . . . . .                                              | 46        |
| 8.3.1    | Testare automată backend . . . . .                                          | 46        |
| 8.3.2    | Testarea convertorului . . . . .                                            | 47        |
| 8.3.3    | Scenarii reprezentative ancorate în teste . . . . .                         | 47        |
| 8.3.4    | Indicatori cantitativi observați . . . . .                                  | 48        |
| 8.3.5    | Testare frontend . . . . .                                                  | 49        |
| 8.4      | Validare funcțională . . . . .                                              | 49        |
| 8.5      | Limitări ale validării actuale . . . . .                                    | 50        |
| 8.6      | Concluzii intermediare . . . . .                                            | 50        |
| <b>9</b> | <b>Concluzii și perspective de dezvoltare</b>                               | <b>51</b> |
| 9.1      | Gradul de îndeplinire a obiectivelor . . . . .                              | 51        |
| 9.2      | Puncte forte ale proiectului . . . . .                                      | 51        |
| 9.3      | Limitări . . . . .                                                          | 51        |
| 9.4      | Perspective de dezvoltare . . . . .                                         | 52        |
| 9.5      | Încheiere . . . . .                                                         | 52        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>A</b> | <b>Exemple tehnice suplimentare</b>                        | <b>53</b> |
| A.1      | Exemplu simplificat de reprezentare intermediară . . . . . | 53        |
| A.2      | Exemple simplificate de cereri către API . . . . .         | 54        |
| A.3      | Exemplu simplificat de cod generat . . . . .               | 54        |
| A.4      | Exemplu simplificat de evenimente de streaming . . . . .   | 55        |
| <b>B</b> | <b>Inventar de endpoint-uri API</b>                        | <b>56</b> |
| B.1      | Observații generale . . . . .                              | 56        |
| B.2      | Endpoint-uri pentru cont și autentificare . . . . .        | 56        |
| B.3      | Endpoint-uri pentru proiecte . . . . .                     | 56        |
| B.4      | Endpoint-uri pentru utilizatori și profiluri . . . . .     | 57        |
| B.5      | Endpoint-uri pentru explore și recomandări . . . . .       | 58        |
| B.6      | Endpoint-uri pentru AI și conversie . . . . .              | 58        |
|          | <b>Bibliografie</b>                                        | <b>59</b> |

## Listă de figuri

|     |                                                                                     |    |
|-----|-------------------------------------------------------------------------------------|----|
| 3.1 | Vedere simplificată asupra stack-ului tehnologic folosit de Favigon . . . . .       | 12 |
| 4.1 | Diagrama de use-case simplificată pentru funcționalitățile principale din Favigon   | 15 |
| 4.2 | Arhitectura logică generală a sistemului . . . . .                                  | 16 |
| 4.3 | Vedere de tip clean architecture pentru backend-ul Favigon . . . . .                | 17 |
| 4.4 | Model conceptual simplificat al datelor principale . . . . .                        | 18 |
| 4.5 | ERD simplificat pentru schema fizică a bazei de date . . . . .                      | 18 |
| 5.1 | Interfața paginii Explore, cu proiecte publice și utilizatori sugerați . . . . .    | 25 |
| 5.2 | Vedere structurală simplificată a principalelor clase și servicii backend . . . . . | 26 |
| 5.3 | Interfața editorului canvas în timpul editării unui proiect . . . . .               | 27 |
| 5.4 | Pagina de preview folosită pentru vizualizarea rezultatului generat . . . . .       | 28 |
| 5.5 | Servicii principale implicate în funcționarea editorului canvas . . . . .           | 29 |
| 6.1 | Pipeline-ul AI în trei faze folosit în Favigon . . . . .                            | 32 |
| 6.2 | Diagrama de secvență simplificată pentru fluxul AI cu streaming . . . . .           | 35 |
| 6.3 | Exemplu de artefacte generate și inspectate în interfața de export . . . . .        | 37 |
| 6.4 | Fluxul design-to-code bazat pe reprezentarea intermediară . . . . .                 | 38 |
| 8.1 | Straturi complementare de validare folosite în proiect . . . . .                    | 50 |

## Listă de tabele

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 2.1 | Comparație între Favigon și aplicații relevante . . . . .                   | 6  |
| 3.1 | Servicii și volume definite în orchestrarea locală a proiectului . . . . .  | 11 |
| 4.1 | Trasabilitatea dintre obiectivele proiectului, module și validare . . . . . | 14 |
| 5.1 | Rute frontend principale și rolul lor în aplicație . . . . .                | 22 |
| 5.2 | Controllere API și responsabilitățile lor dominante . . . . .               | 25 |
| 6.1 | Câmpuri relevante din cererile AI folosite de backend . . . . .             | 34 |
| 8.1 | Măsuri de securitate implementate în Favigon . . . . .                      | 46 |
| 8.2 | Distribuția testelor automate backend . . . . .                             | 47 |
| 8.3 | Scenarii reprezentative verificate prin teste automate . . . . .            | 48 |
| 8.4 | Indicatori cantitativi observați în validarea curentă . . . . .             | 48 |
| B.1 | Endpoint-uri principale din zona de autentificare . . . . .                 | 56 |
| B.2 | Endpoint-uri principale pentru proiecte . . . . .                           | 57 |
| B.3 | Endpoint-uri principale pentru utilizatori . . . . .                        | 57 |
| B.4 | Endpoint-uri din zona explore . . . . .                                     | 58 |
| B.5 | Endpoint-uri pentru AI și converter . . . . .                               | 58 |



# Capitolul 1

## Introducere

### 1.1 Contextul actual al temei

În dezvoltarea produselor digitale, partea de design și partea de implementare continuă să fie tratate, de multe ori, ca etape separate. Designerul lucrează cu layout, ierarhie vizuală și consistență, în timp ce dezvoltatorul lucrează cu componente, stare, apeluri HTTP, validări și constrângeri de framework. În practică, diferența dintre aceste două moduri de lucru introduce timp suplimentar și suficiente ocazii pentru neînțelegeri.

Într-o aplicație web modernă de tip SPA, interfața nu mai este doar o succesiune de pagini statice. Ea depinde de rutare client-side, de stări locale, de interacțiuni asincrone cu backend-ul și de actualizări frecvente ale datelor [1]. Din acest motiv, proiectarea și implementarea sunt mult mai apropiate decât par la prima vedere. Nu este suficient ca un ecran să arate bine; el trebuie să poată fi salvat, refolosit, validat și transformat într-un rezultat executabil.

În paralel, apar tot mai multe instrumente care încearcă să scurteze drumul dintre idee și rezultat: editoare vizuale, aplicații no-code, sisteme de handoff sau generatoare bazate pe AI. Totuși, în multe cazuri, aceste produse rezolvă doar o bucată din problemă. Unele sunt bune pentru design, dar nu pentru export tehnic. Altele generează rapid ceva vizual, dar oferă puțin control asupra structurii interne. În cadrul acestei lucrări, punctul de plecare a fost tocmai această fragmentare.

### 1.2 Problema pe care o urmărește lucrarea

Problema urmărită este concretă: cum poate fi redusă distanța dintre descrierea unei interfețe, editarea ei vizuală și codul care ajunge să ruleze în browser? În fluxul obișnuit, utilizatorul descrie o nevoie, designul este realizat într-un instrument separat, iar implementarea este refăcută manual de la zero. Fiecare trecere dintre etape consumă timp și poate introduce diferențe față de intenția inițială.

Din această observație rezultă trei subprobleme. Prima este legată de editarea vizuală: utilizatorul are nevoie de un spațiu de lucru în care să poată construi și modifica o interfață fără să intre direct în cod. A doua este problema reprezentării interne: dacă se dorește folosirea AI-ului și exportul multi-framework, nu este suficient un rezultat care arată bine, ci este necesar un model de date coerent și validabil. A treia este problema reutilizării: odată creat, un proiect ar trebui să poată fi publicat, reluat și adaptat de alți utilizatori.

Favigon a fost gândit ca răspuns la aceste trei nevoi. Platforma propune un editor vizual propriu, o reprezentare intermediară a interfeței, generare asistată de AI și export în cod, toate integrate în același produs. Accentul nu cade pe publicarea imediată a unui site, ci pe construirea unui flux de lucru coerent, în care designul poate fi creat, ajustat, validat și convertit într-o formă tehnică reutilizabilă.

### 1.3 Motivația alegerii temei

Tema a fost aleasă deoarece permite abordarea simultană a mai multor direcții relevante pentru o lucrare de licență. În primul rând, are o componentă clară de arhitectură software. Un produs ca Favigon nu poate exista doar ca interfață sau doar ca API, pentru că valoarea lui

rezultă din cooperarea mai multor subsisteme: autentificare, proiecte, editor, AI, conversie și zonă socială.

În al doilea rând, tema are o componentă puternică de experiență a utilizatorului. Editorul canvas trebuie să fie suficient de fluid încât să permită experimentare reală, nu doar demonstrații scurte. Asta înseamnă selecție, zoom, istoric, clipboard, autosave, previzualizare și suport pentru mai multe pagini. O parte importantă din dificultatea proiectului a venit tocmai din faptul că aceste comportamente trebuie să funcționeze împreună.

În al treilea rând, integrarea AI-ului într-un produs real este un subiect actual, dar util doar dacă este tratat pragmatic. În această lucrare, AI-ul nu este folosit ca element decorativ și nici ca generator opac de cod, ci ca parte a unui flux cu etape, reguli și validare explicită prin JSON Schema [32, 17, 24]. Tocmai această abordare face tema potrivită pentru o analiză inginerască serioasă.

## 1.4 Obiectivele proiectului

Obiectivul general al proiectului este realizarea unei platforme web care să permită:

- crearea și editarea vizuală a interfețelor web într-un canvas;
- generarea asistată de AI a unei structuri inițiale pentru pagini;
- folosirea unei reprezentări intermediare comune pentru editare și export;
- exportul rezultatului în HTML, React și Angular;
- publicarea și reutilizarea proiectelor într-un spațiu cu funcții sociale.

Pentru atingerea acestui obiectiv au fost urmărite și câteva obiective specifice:

1. proiectarea unei arhitecturi full-stack clare, cu separare între API, logică de aplicație, infrastructură și convertor;
2. definirea unui model de date care să acopere utilizatorii, proiectele, relațiile sociale și autentificarea externă;
3. dezvoltarea unui editor canvas cu suport pentru mai multe pagini și operații uzuale de editare;
4. introducerea unei reprezentări intermediare reutilizabile de către editor, AI și motorul de conversie;
5. implementarea unui pipeline AI în mai multe faze, pentru a crește controlul asupra rezultatului;
6. validarea soluției prin teste automate și scenarii funcționale.

## 1.5 Contribuțiile proprii

Aportul personal se reflectă în principal în următoarele direcții:

1. integrarea într-un singur produs a editorului vizual, a generării AI, a conversiei în cod și a funcțiilor sociale;
2. organizarea editorului canvas pe servicii specializate pentru viewport, istoric, clipboard, gesturi și persistare;
3. definirea unei reprezentări intermediare a interfeței, folosită atât la editare, cât și la export;
4. implementarea unui pipeline AI în trei faze: intenție, structură și stilizare;
5. dezvoltarea unui motor de conversie capabil să genereze HTML, React și Angular, inclusiv pentru scenarii multi-page;

6. completarea părții tehnice cu funcții reale de produs: autentificare, 2FA, profiluri, proiecte publice și mecanisme de reutilizare.

Importanța acestor contribuții nu stă doar în existența lor individuală, ci în felul în care se leagă între ele. Fără modelul intermediar, AI-ul și exportul ar fi mult mai greu de controlat. Fără editorul canvas, platforma ar rămâne la nivelul unui generator. Fără partea de produs, aplicația ar fi doar un demo tehnic.

## 1.6 Metoda de lucru

Dezvoltarea aplicației a fost iterativă. Mai întâi au fost implementate funcțiile de bază necesare pentru existența produsului: autentificare, proiecte și un editor minimal. După aceea, au fost adăugate treptat persistarea automată, previzualizarea, exportul și comportamentele mai complexe din editor. Pipeline-ul AI a fost integrat abia după stabilizarea modelului intern, tocmai pentru a evita generarea unor rezultate greu de folosit mai departe.

Pe partea de backend a fost urmărită o organizare apropiată de principiile prezentate în literatura de arhitectură software [5, 19]. Asta a însemnat separarea controllerelor de serviciile de aplicație și separarea logicii de business de infrastructură. Nu a fost urmărită o implementare dogmatică, ci una suficient de clară încât proiectul să poată fi extins fără a deveni greu de urmărit.

Pe partea de frontend, decizia importantă a fost păstrarea logicii specifice editorului în interiorul feature-ului ‘canvas’, nu dispersarea ei în zone generice. Cum editorul concentrează cea mai mare parte a complexității aplicației, această alegere a ajutat la păstrarea contextului tehnic aproape de ecranele și serviciile care îl folosesc.

## 1.7 Structura lucrării

Lucrarea este organizată în nouă capitole principale, la care se adaugă rezumatul și două anexe.

Capitolul al doilea analizează domeniul și poziționează proiectul față de aplicații relevante. Capitolul al treilea prezintă fundamentele teoretice și tehnologiile folosite. Capitolul al patrulea tratează analiza cerințelor și proiectarea sistemului. Capitolul al cincilea descrie implementarea platformei, cu accent pe modulele și fluxurile principale. Capitolul al șaselea este dedicat părții de AI și conversiei din design în cod, adică nucleului tehnic al lucrării. Capitolul al șaptelea urmărește scenariile de utilizare și o evaluare practică a produsului. Capitolul al optulea discută securitatea, testarea și validarea. Ultimul capitol sintetizează rezultatele obținute, limitele curente și direcțiile de continuare.

Anexele completează partea principală cu exemple tehnice suplimentare și cu o vedere de ansamblu asupra API-ului. Prin această structură, partea teoretică, partea de proiectare și contribuțiile practice rămân separate suficient de clar, fără să se piardă legătura dintre ele.

## Capitolul 2

# Analiza domeniului și studiu comparativ

## 2.1 Evoluția spațiului design-to-code

Zona de *design-to-code* nu este nouă, dar a devenit mult mai vizibilă odată cu maturizarea aplicațiilor web și cu apariția modelelor generative moderne. În literatura mai veche, problema era formulată deseori ca transformarea unei descrieri vizuale într-un cod de interfață. Un exemplu cunoscut este *pix2code*, unde autorul tratează generarea codului din capturi de ecran ale interfeței [6]. Mai recent, lucrările dedicate modelelor mari de limbaj și modelelor multimodale arată că interesul s-a mutat de la simpla reconstrucție a aspectului spre păstrarea structurii, a layout-ului și a intenției de produs, inclusiv prin abordări care împart interfața în segmente mai mici pentru a controla mai bine fidelitatea rezultatului [28, 30, 33].

În practică, piața a evoluat într-o direcție puțin diferită de lucrările academice. Produsele comerciale au observat că utilizatorul nu are nevoie doar de un model care produce cod, ci de un întreg flux de lucru: proiectare, organizare pe pagini, colaborare, verificare, export, publicare și iterație. Din acest motiv, instrumentele moderne nu mai pot fi analizate doar prin întrebarea “generează sau nu cod?”, ci prin ansamblul de funcții pe care îl oferă.

Un aspect care apare repede în practică este următorul: nu contează doar dacă se poate genera ceva, ci și dacă rezultatul poate fi integrat ușor într-un produs real. Un layout poate arăta bine într-o demonstrație, dar dacă nu are o structură logică, nu respectă constrângeri tehnice și nu poate fi ajustat prin iterații mici, valoarea lui scade. De aici a pornit și decizia din Favigon de a nu trata AI-ul ca pe un generator final, ci ca pe un pas dintr-un flux controlat, sprijinit de reprezentarea intermediară și de validare.

## 2.2 Criterii de comparație

Pentru a poziționa corect proiectul, au fost alese criterii relevante atât din punct de vedere tehnic, cât și din punct de vedere al utilității pentru utilizator:

- **editare vizuală directă:** existența unui spațiu de lucru de tip canvas sau editor vizual;
- **asistență AI:** posibilitatea de a porni de la un prompt, sugestii sau generare automată de structură;
- **design-to-code:** existența unei conversii către cod sau măcar a unor artefacte pentru handoff tehnic;
- **orientare spre produs final:** suport pentru publicare, organizare pe pagini sau dezvoltare continuă;
- **partajare și colaborare:** mecanisme prin care rezultatele pot fi distribuite sau reutilizate;
- **control tehnic al ieșirii:** cât de mult poate fi influențată structura rezultatului și cât de ușor poate fi integrat în cod existent.

Aceste criterii nu sunt perfecte și nu încearcă să stabilească “cel mai bun” produs. Scopul lor este mai modest: să arate unde se află Favigon pe o hartă a instrumentelor existente și ce gol încearcă să acopere.

## 2.3 Aplicații relevante din aceeași zonă

### 2.3.1 Figma

Figma este un punct de referință pentru orice discuție despre proiectarea interfețelor. Din perspectiva acestei lucrări, interesul nu este dat doar de partea de design colaborativ, ci și de Dev Mode. Documentația oficială arată clar că Dev Mode este gândit ca spațiu pentru inspectarea designurilor și pentru apropierea dintre design și implementare [15]. Cu toate acestea, Figma rămâne în primul rând un instrument de design și handoff. Chiar dacă poate genera fragmente de cod sau poate conecta design system-uri la bazele de cod, el nu devine automat un mediu complet de construire și publicare a produsului.

Pentru comparația cu Favigon, Figma este util în două direcții. Prima este ideea de structură vizuală clară și multi-page. A doua este faptul că handoff-ul către dezvoltare nu este suficient în sine; dacă se urmăresc iterații rapide, este necesar un spațiu în care designul să poată fi nu doar inspectat, ci și transformat mai departe în ceva executabil.

### 2.3.2 Webflow

Webflow este relevant deoarece reprezintă foarte bine zona de *visual website builder*. Pagina oficială pune accent pe construcția vizuală, publicare și generare rapidă de site-uri [31]. Avantajul evident al acestei abordări este că rezultatul ajunge repede online și este ușor de gestionat din punct de vedere al conținutului. În schimb, Webflow are un focus clar pe zona de website publishing și CMS, nu pe un model general de interfață reutilizabil între mai multe framework-uri.

Pentru poziționarea Favigon, Webflow este important în special pentru a delimita intenția aplicației. Nu a fost urmărită construirea încă unui publicator vizual de site-uri. Accentul cade mai mult pe controlul tehnic al structurii, exportul în cod și legătura dintre un editor vizual și o reprezentare internă care poate fi convertită.

### 2.3.3 Framer

Framer este interesant fiindcă mută discuția mai aproape de zona AI. Pagina oficială pentru Wireframer arată o abordare în care un prompt este folosit pentru a construi rapid un layout responsive, cu accent pe structură și flux [16]. Este o direcție apropiată de cea urmărită și aici, însă diferența majoră este că în Favigon este făcută explicită separarea dintre intenție, structură și stilizare.

Cu alte cuvinte, Framer demonstrează că generarea ghidată de prompt poate fi utilă pentru pornirea rapidă a unui proiect. Totuși, pentru o lucrare inginerescă este mai valoros să fie explicați și controlați pașii interni ai generării, nu doar să fie observat rezultatul final.

### 2.3.4 Builder.io

Builder.io este probabil cel mai apropiat reper comercial pentru direcția urmărită în această lucrare. Platforma lor pune accent pe dezvoltare vizuală, AI și reutilizarea componentelor existente [8, 9]. Ideea de a nu porni de la zero de fiecare dată, ci de a avea o punte între design, componente și cod, este una foarte relevantă.

Diferența principală este că Builder.io este orientat puternic spre integrarea într-un ecosistem existent și spre productivitate în contexte enterprise. Favigon, în schimb, este un proiect academic și experimental, în care accentul este pus pe claritatea modelului intern și pe trasabilitatea tehnică a fiecărei etape. Asta înseamnă că produsul nu concurează direct cu Builder.io la nivel de maturitate comercială, dar tratează o parte din probleme într-un mod mai transparent pentru analiză și explicație.

## 2.4 Tabel comparativ

**Tabela 2.1:** Comparație între Favigon și aplicații relevante

| Criteriu                                   | Figma       | Webflow   | Framer  | Builder.io | Favigon |
|--------------------------------------------|-------------|-----------|---------|------------|---------|
| Editor vizual direct                       | Da          | Da        | Da      | Da         | Da      |
| Pornire din prompt AI                      | Parțial     | Da        | Da      | Da         | Da      |
| Model intern pentru export multi-framework | Nu explicit | Nu        | Limitat | Parțial    | Da      |
| Export HTML, React, Angular                | Nu direct   | Nu direct | Limitat | Integrare  | Da      |
| Funcții sociale în aceeași platformă       | Nu          | Nu        | Nu      | Nu         | Da      |
| Publicare proiecte proprii                 | Parțial     | Da        | Da      | Da         | Parțial |
| Reutilizare prin fork / star / like        | Nu          | Nu        | Nu      | Nu         | Da      |
| Accent pe controlul etapelor AI            | Nu          | Nu        | Parțial | Parțial    | Da      |

Tabelul 2.1 nu are rolul de a demonstra că Favigon este “mai bun” decât produsele consacrate. Ar fi o comparație nerealistă. Scopul lui este să arate că Favigon combină, într-o formă academică și experimentală, elemente care în alte produse apar separate: editor vizual, asistență AI, export în cod și funcții sociale.

## 2.5 Poziționarea proiectului Favigon

În urma comparației, Favigon poate fi poziționat ca o platformă experimentală de tip editor vizual asistat de AI, orientată mai puțin spre publicare imediată și mai mult spre explorarea unui flux coerent între idei, design, model intern și cod executabil. Cu alte cuvinte, produsul nu își propune să concureze prin maturitatea ecosistemului, ci prin claritatea traseului tehnic pe care îl oferă.

În termeni practici, produsul este mai potrivit pentru prototipare rapidă, experimente de interfață, proiecte academice și aplicații mici sau medii decât pentru scenarii enterprise foarte rigide. Această delimitare explică mai bine valoarea reală a platformei: nu promite înlocuirea unui ecosistem comercial matur, ci oferă un cadru integrat pentru a trece mai repede de la idee la un rezultat executabil și ușor de analizat tehnic.

Această poziționare vine și cu limite. De exemplu, Favigon nu are nivelul de rafinament vizual sau de integrare enterprise al unor produse comerciale mature. Nici partea de colaborare simultană în timp real nu este încă implementată. Totuși, pentru scopul lucrării, aceste limite nu sunt un dezavantaj major. Ele permit concentrarea pe întrebările urmărite aici:

- cum pot reprezenta o interfață într-o formă suficient de stabilă încât să o editez și să o export;
- cum pot integra AI într-un mod controlabil;
- cum pot susține reutilizarea prin funcții sociale simple;
- cum pot organiza întregul produs astfel încât să rămână clar tehnic.

## 2.6 Golul pe care îl acoperă Favigon

Din analiza de mai sus rezultă că există un spațiu intermediar între instrumentele clasice de design, generatoarele AI rapide și platformele orientate strict spre publicare. Favigon încearcă să acopere tocmai acest spațiu intermediar. Mai exact, platforma propune:

1. un editor propriu, nu doar o zonă de import sau inspectare;
2. un model intermediar explicit, nu doar un rezultat final opac;
3. export multi-framework, nu doar un cod specific unei singure ținte;
4. funcții sociale integrate, astfel încât proiectele să poată circula în comunitate;
5. o integrare a AI-ului care poate fi analizată, validată și îmbunătățită pas cu pas.

Din punct de vedere academic, aceasta este o direcție relevantă pentru o lucrare de licență. Proiectul nu se reduce la folosirea unui model AI pentru obținerea unor pagini, ci pune în discuție mai multe aspecte ingineresti: modelare internă, arhitectură pe straturi, validare, testare, UX și decizii de produs.

## 2.7 Concluzii intermediare

Analiza domeniului arată că proiectul Favigon nu apare într-un vid, ci într-un context în care instrumentele pentru design, dezvoltare vizuală și AI cresc rapid. În același timp, ea arată și de ce proiectul are sens ca temă de licență. Există deja instrumente puternice, dar nu toate tratează împreună editarea vizuală, generarea controlată, exportul tehnic și partajarea în aceeași platformă.

Concluzia acestui capitol este că Favigon nu trebuie evaluat ca un produs comercial finalizat, ci ca o platformă software care încearcă să demonstreze fezabilitatea unui flux integrat. Pe baza acestei concluzii, capitolul următor trece de la piață și comparații la fundamentele teoretice și tehnologiile pe care s-a sprijinit implementarea.

## Capitolul 3

# Fundamente teoretice și tehnologii utilizate

### 3.1 Considerații generale

Înainte de a descrie implementarea concretă, este utilă clarificarea fundamentelor pe care se sprijină proiectul. Favigon nu reunește doar câteva ecrane și endpoint-uri, ci mai multe idei care trebuie să funcționeze împreună: aplicație single-page, API REST, autentificare pe bază de token, persistare relațională, orchestrare containerizată, generare asistată cu ieșiri structurate și conversie dintr-un model intermediar către cod executabil.

Aceste noțiuni sunt tratate împreună deoarece în proiect ele nu apar izolat. De exemplu, editorul canvas este implementat pe frontend, dar depinde direct de modul în care proiectele sunt persistate în backend. Pipeline-ul AI produce o structură intermediară, dar valoarea ei reală se vede abia când motorul de conversie o transformă în fișiere HTML, React sau Angular. Din acest motiv, capitolul de față urmărește să explice nu doar *ce* tehnologii sunt folosite, ci și *de ce* alegerea lor a fost potrivită pentru problema rezolvată.

### 3.2 Aplicații single-page și rolul frontend-ului modern

Favigon folosește pe partea de client o aplicație Angular 20 organizată sub forma unui SPA. Într-un SPA, browserul încarcă inițial o singură pagină, iar apoi navigarea între ecrane este gestionată în principal client-side, fără reîncărcarea completă a documentului [1]. Acest model este potrivit pentru Favigon din două motive.

Primul motiv este experiența utilizatorului. Editorul canvas presupune interacțiuni dese, actualizări vizuale rapide și trecere fluidă între zone precum profil, explore, settings, proiect și preview. Dacă fiecare navigare ar depinde de reîncărcarea integrală a paginii, aplicația ar deveni mai greu de folosit și mai puțin apropiată de instrumentele moderne de design.

Al doilea motiv este organizarea tehnică. Angular oferă rutare, componente, injecție de dependențe, gestiune a formularelor și integrare cu HTTP într-un ecosistem unitar. În plus, proiectul folosește componente standalone și ‘provideRouter’, ceea ce simplifică structura aplicației și reduce dependența de module tradiționale [2, 4]. Pentru un proiect în care cea mai mare complexitate se află în interfață, această coerență a framework-ului a contat mult.

Un element important în implementare este folosirea *signals*. Documentația Angular prezintă signals ca mecanism reactiv granular pentru urmărirea și actualizarea stării [3]. În editorul canvas, această abordare este utilă deoarece permite reacții mai fine la schimbarea selecției, a viewport-ului, a paginii curente sau a istoricului, fără a introduce un strat extern de management al stării. Pentru un feature foarte interactiv, această soluție oferă un raport bun între simplitate și control.

### 3.3 Arhitectura client-server și API-ul REST

Comunicarea dintre frontend și backend este organizată sub forma unui API REST. În literatura clasică, stilul REST este tratat ca o modalitate de proiectare a sistemelor software bazate pe resurse, interacțiuni standardizate și separarea clară între client și server [14]. În Favigon, această alegere a fost una pragmatică.



Aplicația are resurse destul de clar delimitate: utilizatori, proiecte, design-uri, upload-uri, rezultate generate, relații sociale și operații de autentificare. Pentru fiecare dintre acestea a fost naturală definirea unor controllere și endpoint-uri specializate. Spre exemplu, proiectele sunt gestionate prin endpoint-uri dedicate pentru creare, actualizare, design, thumbnail, assets, like, star sau fork, în timp ce autentificarea este separată în propriul controller.

ASP.NET Core 9 a fost ales pentru backend deoarece oferă un cadru potrivit pentru API-uri moderne: rutare declarativă, middleware configurabil, DI integrat, autentificare, autorizare și extensibilitate la nivel de pipeline HTTP. Dincolo de faptul că framework-ul este matur, el permite și o separare curată între straturile aplicației. În proiect, API-ul doar primește cereri și returnează răspunsuri, în timp ce logica de business este ținută în servicii de aplicație, iar persistența și integrarea externă sunt mutate în infrastructură.

### 3.4 Autentificare, token-uri și securitate la nivel de cerere

Zona de autentificare din Favigon combină mai multe mecanisme: login și register clasic, OAuth cu furnizori externi, refresh token și autentificare în doi pași. Din perspectiva acestei lucrări, important este modul în care ele sunt integrate coerent.

ASP.NET Core oferă suport direct pentru autentificare JWT bearer [7]. În proiect, token-ul de acces este transportat prin cookie HTTP only, iar backend-ul îl extrage din cookie-ul 'jwt' în cadrul configurării 'JwtBearer'. Practic, acest lucru combină avantajul token-ului auto-conținut cu avantajul unui transport mai greu de accesat din JavaScript. Această alegere nu elimină orice risc posibil, dar este rezonabilă pentru tipul de aplicație construit aici.

În plus, proiectul folosește refresh token separat și limitează calea cookie-ului de refresh la endpoint-ul de reînnoire, ceea ce reduce suprafața de expunere. Pentru operații sensibile sau scenarii de risc mai mare, este introdusă și autentificare în doi pași pe bază de cod temporar transmis prin email. Din perspectiva ghidurilor OWASP, folosirea unui factor suplimentar este una dintre cele mai eficiente măsuri pentru reducerea atacurilor care pornesc de la compromiterea parolei [25, 26].

Dincolo de autentificare, backend-ul folosește și rate limiting. Documentația ASP.NET Core tratează rate limiting-ul ca mecanism de control al fluxului de cereri, util pentru protecție împotriva abuzului și pentru stabilitate [18]. În proiect sunt definite politici diferite pentru zone precum autentificare, AI, converter și users, tocmai pentru că profilul de risc și costul operațiilor nu sunt aceleași.

### 3.5 Reprezentarea intermediară a interfeței

Una dintre ideile de bază ale proiectului este că designul nu trebuie legat direct nici de canvas, nici de HTML final. Între aceste două niveluri este introdusă o reprezentare intermediară bazată pe noduri ('IRNode'), layout, stil, poziționare, metadata și variante responsive. Din punct de vedere conceptual, această reprezentare joacă rolul unui limbaj intern al aplicației.

Avantajul principal al unui astfel de model este separarea preocupărilor. Editorul poate să manipuleze elemente și structuri fără să producă imediat cod pentru browser. În același timp, motorul de conversie poate genera HTML, React sau Angular fără să depindă de detaliile de implementare ale editorului. Asta înseamnă că aceeași structură poate fi folosită în două scopuri diferite: editare și export.

În plus, modelul intermediar permite validare înainte de generare. Dacă un arbore de noduri este incomplet, inconsistent sau imposibil de randat corect, problema poate fi detectată înainte de etapa de export. Acesta este un avantaj important față de abordările care merg direct

din prompt în cod. În Favigon, această validare intermediară face posibilă combinarea AI-ului cu un flux tehnic controlabil.

### 3.6 JSON Schema și ieșiri structurate pentru AI

În momentul în care s-a decis folosirea unui model AI pentru generarea interfețelor, una dintre primele probleme a fost fiabilitatea formatului de ieșire. Un răspuns foarte bun ca limbaj natural nu este automat util pentru o aplicație software. Este nevoie de date structurate, de câmpuri verificabile și de o formă care să poată fi procesată programatic.

Pentru această nevoie, JSON Schema este un instrument foarte potrivit. Specificația definește un mod standardizat de a exprima structura, tipurile și restricțiile unui document JSON [32, 17]. În Favigon, schema este folosită pentru a constrânge ieșirile modelului AI atunci când se generează atât blueprint-ul intenției, cât și arborele intermediar al interfeței.

Din perspectiva OpenAI API, această idee este compatibilă cu *structured outputs*, adică solicitarea unui răspuns care respectă o anumită schemă [24, 23]. Totuși, faptul că formatul este corect nu garantează automat că și conținutul este potrivit. De aceea, în implementare nu s-a mers doar până la schema JSON, ci au fost adăugate și validări semantice, plus un mecanism de reparare atunci când modelul produce ieșiri invalide sau incomplete.

### 3.7 Baza de date relațională și persistența

Pentru persistență este folosit PostgreSQL, prin Entity Framework Core și providerul Npgsql [27, 22]. Alegerea unei baze de date relaționale a fost firească, deoarece datele principale au relații clare: un utilizator poate avea mai multe proiecte, proiectele pot avea aprecieri și bookmark-uri, utilizatorii se pot urmări între ei, iar un proiect poate fi fork-uit din alt proiect.

Pe lângă modelul relațional clasic, aplicația păstrează și un câmp de tip JSON serializat pentru designul proiectului. Aici există un compromis asumat. Pe de o parte, datele vizuale ale canvas-ului sunt prea flexibile pentru a fi modelate comod în tabele relaționale fine. Pe de altă parte, utilizatorii, relațiile sociale și metadatele proiectelor se potrivesc foarte bine într-un model relațional. În practică, combinația dintre tabele relaționale și document JSON pentru design s-a dovedit suficient de echilibrată pentru scopul proiectului.

În implementarea concretă, schema rezultată este compactă, dar suficient de expresivă pentru produsul construit. Baza de date conține tabelele `users`, `projects`, `linked_accounts`, `user_follows`, `project_bookmarks` și `project_likes`. Tabelele relaționale descriu identitatea utilizatorului, proiectele, relațiile sociale și interacțiunile dintre utilizatori și proiecte, iar câmpul `DesignJson` din `projects` păstrează documentul de canvas sub formă `jsonb`. Această alegere permite filtrare și agregare eficientă pentru metadate, fără a forța fragmentarea structurii vizuale în numeroase tabele auxiliare.

Schema fizică include și constrângeri relevante pentru consistență. Email-ul utilizatorului este unic, conturile OAuth au unicitate atât pe perechea `(Provider, ProviderUserId)`, cât și pe perechea `(UserId, Provider)`, iar tabelele de tip `follow`, `bookmark` și `like` folosesc chei compuse pentru a preveni dublurile. În plus, proiectele folosesc o autoreferință prin `ForkedFromProjectId`, ceea ce face posibilă trasabilitatea unui fork fără a duplica separat noțiunea de proiect-sursă. Contextul EF Core completează automat timestamp-urile `CreatedAt` și `UpdatedAt`, astfel încât persistența să rămână coerentă și la nivel operațional, nu doar la nivel de structură.

## 3.8 Orchestrare și infrastructură

La nivel de infrastructură, proiectul folosește Docker Compose pentru definirea și pornirea serviciilor principale: baza de date PostgreSQL, API-ul, frontend-ul și tunelul Cloudflare [13, 12]. Această variantă reduce efortul necesar pentru pornirea locală și descrie într-un singur fișier dependențele dintre servicii, volumele persistente și rețeaua internă.

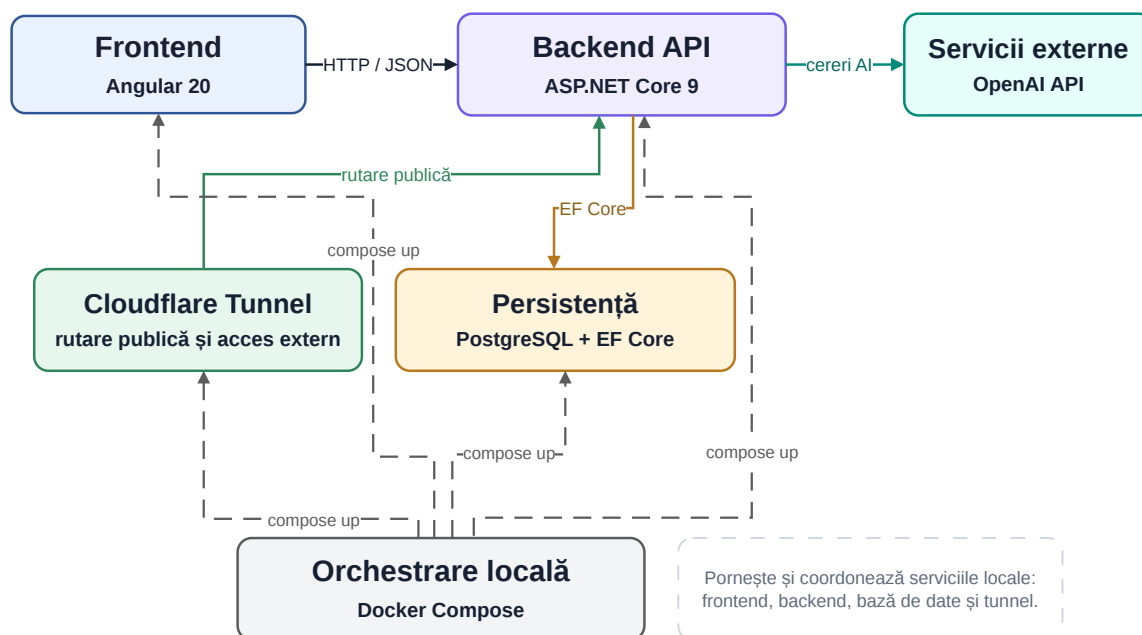
Avantajul principal al acestui mod de lucru este repetabilitatea. Când toate componentele importante sunt definite declarativ, mediul devine mai ușor de refăcut, de demonstrat și de mutat între sisteme. Pentru o lucrare de licență, acest aspect nu este deloc minor, deoarece ușurează demo-ul și reduce riscul ca aplicația să funcționeze doar pe o singură configurație locală.

În configurația actuală, fișierul `docker-compose.yml` nu pornește doar containerele, ci exprimă și câteva decizii operaționale utile. Baza de date are *healthcheck*, astfel încât API-ul să nu pornească înainte ca PostgreSQL să fie pregătit pentru conexiuni. API-ul montează separat volumul pentru upload-uri, ceea ce permite persistența imaginilor și thumbnail-urilor independent de ciclul containerului. Frontend-ul este construit cu argument explicit pentru fișierul de mediu de producție, iar tunelul Cloudflare este atașat peste aplicația deja pornită, nu folosit ca substitut pentru rețeaua internă.

**Tabela 3.1:** Servicii și volume definite în orchestrarea locală a proiectului

| Serviciu / volum | Rol principal                                         | Observații utile                                                              |
|------------------|-------------------------------------------------------|-------------------------------------------------------------------------------|
| db               | instanță PostgreSQL 16 pentru persistența relațională | folosește <i>healthcheck</i> cu <i>pg_isready</i> și volum persistent dedicat |
| api              | aplicația ASP.NET Core expusă intern pe portul 8080   | depinde de sănătatea bazei de date și montează directorul de upload-uri       |
| frontend         | aplicația Angular compilată pentru mediu de producție | depinde de API și rulează în aceeași rețea internă                            |
| cloudflared      | publicare externă controlată prin Cloudflare Tunnel   | folosește token extern și pornește doar după frontend                         |
| postgres_data    | persistă datele PostgreSQL                            | evită pierderea datelor la recrearea containerelor                            |
| uploads_data     | persistă fișierele încărcate și thumbnail-urile       | separă conținutul generat de ciclul de build/deploy                           |

Pentru expunerea aplicației în exterior este folosit și Cloudflare Tunnel. Documentația oficială subliniază faptul că tunelul funcționează prin conexiuni doar outbound dintre infrastructura locală și rețeaua Cloudflare, evitând expunerea directă a unui IP public și reducând suprafața de atac [11]. În proiect, această alegere a fost utilă mai ales pentru publicare și testare externă, fără a complica prea mult configurarea rețelei. Vederea simplificată a stack-ului rezultat este redată în Figura 3.1.



**Figura 3.1:** Vedere simplificată asupra stack-ului tehnologic folosit de Favigon

### 3.9 De ce această combinație tehnologică a fost potrivită

Privită în ansamblu, combinația Angular + ASP.NET Core + PostgreSQL + OpenAI API + Docker Compose nu este una neobișnuită. Tocmai aici stă avantajul ei practic. Sunt folosite tehnologii bine documentate, mature și suficient de separate ca responsabilitate. Angular acoperă interfața și interacțiunea clientului. ASP.NET Core acoperă API-ul și middleware-ul. PostgreSQL acoperă persistența relațională. OpenAI API acoperă generarea asistată. Docker Compose și Cloudflare Tunnel simplifică operaționalizarea.

Această alegere are și un beneficiu pedagogic. Într-o lucrare de licență, prea multe tehnologii neobișnuite riscă să mute accentul de pe soluția inginerească pe complexitatea setării mediului. Aici valoarea nu vine din folosirea unor instrumente rare, ci din modul în care acestea sunt combinate pentru a obține un produs coerent.

### 3.10 Concluzii intermediare

Fundamentele prezentate în acest capitol explică de ce Favigon a fost construit în forma actuală. SPA-ul susține interacțiunea rapidă. API-ul REST și separarea pe straturi susțin claritatea backend-ului. JWT-ul, 2FA și rate limiting-ul susțin securitatea operațională. Reprezentarea intermediară și JSON Schema susțin controlul asupra generării AI și a exportului. PostgreSQL și Docker Compose susțin stabilitatea și portabilitatea.

Pe baza acestor fundamente, capitolul următor trece la analiza și proiectarea sistemului propriu-zis: cerințe, actori, fluxuri, model de date și arhitectură internă.

## Capitolul 4

# Analiza și proiectarea sistemului

### 4.1 Pornirea de la cerințe

Înainte de implementarea funcționalităților principale, a fost necesară separarea cât mai clară a cerințelor sistemului. Pentru acest tip de proiect, riscul cel mai mare este adăugarea multor funcții atractive, dar fără o structură clară. De aceea, analiza a pornit de la întrebarea simplă: ce trebuie să poată face utilizatorul și ce trebuie să poată susține sistemul în spate?

Din perspectiva utilizatorului final, aplicația trebuie să permită crearea unui cont, autentificarea, administrarea profilului, lucrul cu proiecte și folosirea editorului vizual. Din perspectiva fluxului de produs, trebuie să existe și operații precum publicarea unui proiect, explorarea proiectelor altora, aprecierea lor și reutilizarea prin fork. Din perspectiva obiectivului tehnic al lucrării, sistemul trebuie să poată genera designuri cu ajutorul AI și să exporte rezultatul în cod.

Aceste cerințe au fost apoi grupate în două categorii mari: funcționale și nefuncționale.

### 4.2 Cerințe funcționale

Principalele cerințe funcționale identificate pentru Favigon sunt următoarele:

1. utilizatorul poate crea cont, autentifica și ieși din aplicație;
2. utilizatorul poate folosi login clasic sau autentificare prin furnizori externi;
3. utilizatorul poate activa sau dezactiva autentificarea în doi pași;
4. utilizatorul își poate edita profilul și imaginea de profil;
5. utilizatorul poate crea, edita și șterge proiecte;
6. pentru fiecare proiect, utilizatorul poate salva designul și thumbnail-ul asociat;
7. utilizatorul poate edita vizual proiectul într-un canvas cu mai multe pagini;
8. utilizatorul poate genera o structură de design prin AI, inclusiv în regim streaming;
9. utilizatorul poate valida și exporta proiectul în HTML, React sau Angular;
10. utilizatorul poate marca proiectele altora cu like sau star;
11. utilizatorul poate urmări alți utilizatori și poate vedea proiecte recomandate;
12. utilizatorul poate fork-ui proiecte publice și le poate adapta în propriul spațiu.

La nivel de implementare, aceste cerințe se reflectă destul de direct în controllerele și serviciile backend, respectiv în feature-urile frontend. De exemplu, 'AccountController' și 'AuthService' acoperă fluxurile de autentificare, 'ProjectsController' și 'ProjectService' gestionează ciclul de viață al proiectelor, iar 'AiDesignController' împreună cu serviciile AI acoperă generarea asistată. Pentru a păstra legătura dintre obiective, module și validare, Tabelul 4.1 sintetizează corespondențele principale.

### 4.3 Cerințe nefuncționale

Pe lângă cerințele funcționale, proiectul are și câteva cerințe nefuncționale importante:

- **claritate arhitecturală:** logica de business nu trebuie amestecată în controllere;

- **extensibilitate:** exportul către mai multe framework-uri trebuie să fie posibil fără rescrierea editorului;
- **performanță interactivă:** editorul trebuie să rămână fluid pentru operațiile uzuale;
- **siguranță:** autentificarea și operațiile sensibile trebuie protejate rezonabil;
- **persistență robustă:** designul proiectelor trebuie salvat automat și recuperat coerent;
- **portabilitate:** aplicația trebuie să poată fi pornită ușor local și demonstrată într-un mediu apropiat de producție;
- **testabilitate:** cel puțin partea de backend și motorul de conversie trebuie să poată fi verificate prin teste automate.

Aceste cerințe nefuncționale au influențat puternic structura aplicației. Necesitatea de a exporta în mai multe tehnologii a dus aproape inevitabil la introducerea reprezentării intermediare. Necesitatea de a păstra editorul fluid a dus la împărțirea logicii în servicii specializate pe frontend. Necesitatea de a limita amestecul de responsabilități a dus la structurarea backend-ului pe straturi.

**Tabela 4.1:** Trasabilitatea dintre obiectivele proiectului, module și validare

| Obiectiv                                | Module principale                                                  | Formă de validare                                         |
|-----------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------|
| Autentificare și administrare cont      | 'AccountController',<br>'AuthService', zona de profil utilizator   | teste backend și scenarii de login / 2FA                  |
| Editor vizual multi-page                | 'CanvasPage' și serviciile principale ale editorului               | scenarii de editare, autosave și restaurare               |
| Reprezentare intermediară reutilizabilă | mapper-e canvas ↔ IR,<br>'IRNode', 'ConverterService'              | validare IR și teste pentru convertor                     |
| Generare asistată de AI controlabilă    | 'AiDesignController',<br>'AiDesignService',<br>'AiPipelineService' | validare structurală,<br>auto-reparare și scenarii AI     |
| Export multi-framework                  | 'ConverterController',<br>'ConverterService',<br>'ConverterEngine' | teste HTML / React / Angular,<br>responsive și multi-page |
| Componentă socială și reutilizare       | 'ProjectsController',<br>'UsersController',<br>'ExploreController' | teste service/controller și<br>scenarii sociale           |

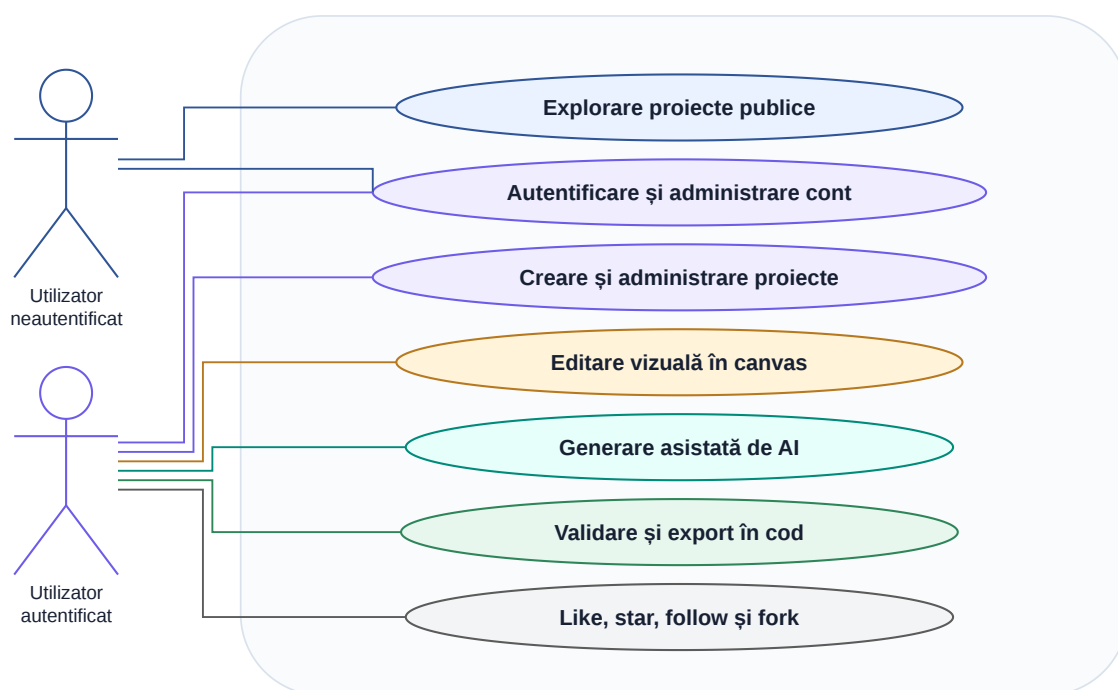
## 4.4 Actorii principali

Sistemul are două categorii principale de actori.

**Utilizatorul neautentificat** poate accesa paginile publice, poate vedea proiectele expuse și poate explora conținutul public. El nu poate modifica proiecte și nu poate folosi editorul complet.

**Utilizatorul autentificat** reprezintă actorul central al sistemului. Acesta își poate administra profilul, își poate crea și edita proiectele, poate folosi editorul canvas, poate apela generarea AI, poate exporta cod și poate interacționa social cu alți utilizatori.

Pentru a sintetiza relația dintre actori și funcționalitățile dominante, Figura 4.1 prezintă o vedere de tip use-case. Ea nu urmărește toate endpoint-urile sau toate stările posibile, ci capabilitățile care structurează produsul la nivel de utilizare.



**Figura 4.1:** Diagrama de use-case simplificată pentru funcționalitățile principale din Favigon

## 4.5 Scenarii principale de utilizare

### 4.5.1 Scenariul 1: creare și editare proiect

Utilizatorul se autentifică, creează un proiect nou și ajunge în editorul canvas. Acolo poate adăuga elemente, poate crea pagini noi, poate modifica proprietăți și poate salva implicit progresul. La ieșire sau la schimbarea paginii, sistemul trebuie să se asigure că starea relevantă este persistată.

### 4.5.2 Scenariul 2: generare asistată de AI

Utilizatorul pornește de la un prompt și solicită generarea unei interfețe. Backend-ul trimite cererea către serviciul AI, primește etapele pipeline-ului și returnează rezultatul. Frontend-ul transformă structura obținută în elemente de canvas și permite utilizatorului să continue editarea manuală. Acest scenariu este important deoarece arată că AI-ul este integrat în fluxul editorului, nu separat de el.

### 4.5.3 Scenariul 3: export în cod

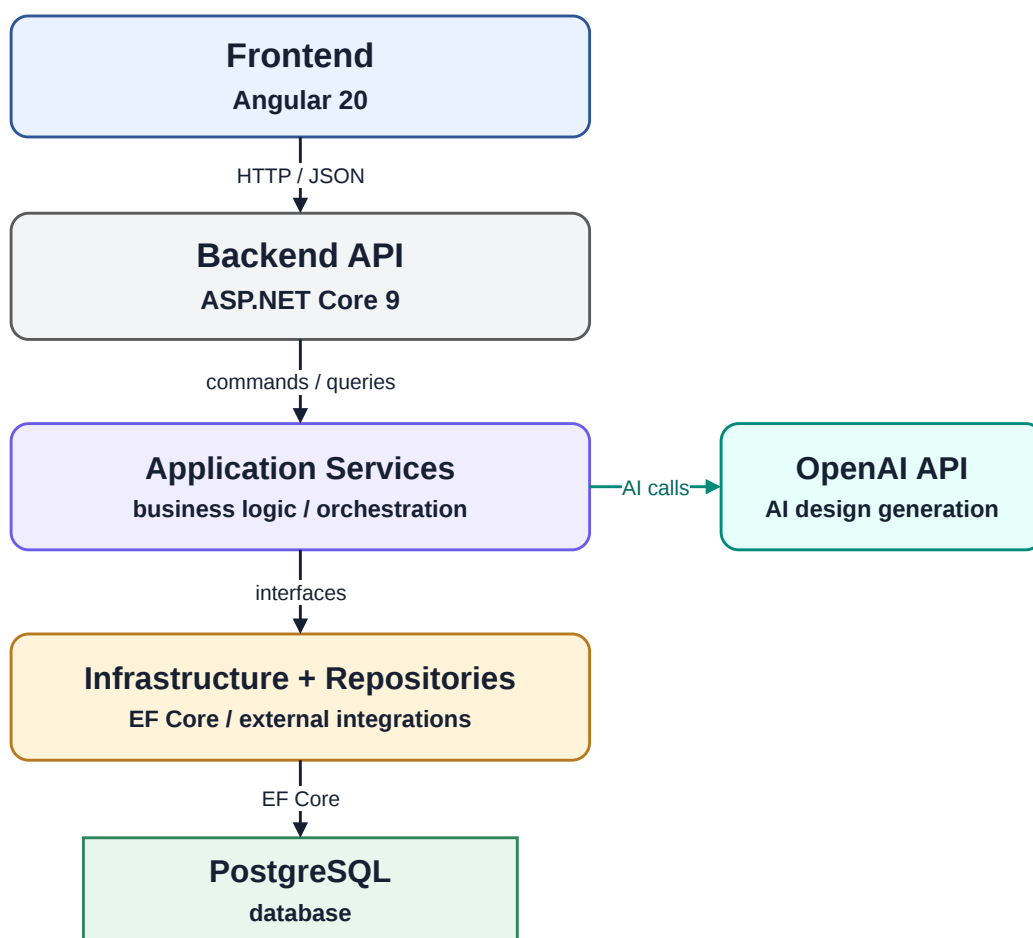
După ce proiectul ajunge într-o stare satisfăcătoare, utilizatorul cere validarea și exportul. Designul este convertit în reprezentarea intermediară, verificat și transmis motorului de conversie. Rezultatul este un set de fișiere pentru framework-ul ales. Acest scenariu justifică existența convertorului ca modul separat.

#### 4.5.4 Scenariul 4: distribuire și reutilizare

Un utilizator publică un proiect, iar alt utilizator îl descoperă în explore sau pe profilul autorului. Poate să îl aprecieze, să îl salveze sau să îl fork-uiască. În acest fel, proiectul capătă și o dimensiune de comunitate, nu rămâne doar un document privat.

### 4.6 Arhitectura generală

La nivel înalt, sistemul este împărțit în frontend, backend, bază de date și servicii externe. Frontend-ul Angular gestionează interfața, rutarea și starea locală a editorului. Backend-ul ASP.NET Core expune endpoint-uri REST și coordonează logica de aplicație. PostgreSQL stochează datele persistente. Serviciile externe includ OpenAI API pentru generare și Cloudflare Tunnel pentru expunere controlată. Arhitectura logică rezultată este redată în Figura 4.2.



**Figura 4.2:** Arhitectura logică generală a sistemului

Această împărțire nu este doar una de prezentare. Ea se regăsește concret în structura soluției backend: proiectele 'Favigon.API', 'Favigon.Application', 'Favigon.Infrastructure', 'Favigon.Domain', 'Favigon.Converter' și 'Favigon.Tests'. Fiecare are un rol clar, iar motorul de conversie este separat explicit deoarece are propriile reguli și propria logică de transformare.

### 4.7 Organizarea backend-ului pe straturi

La nivel de backend, fluxul tipic al unei cereri este:

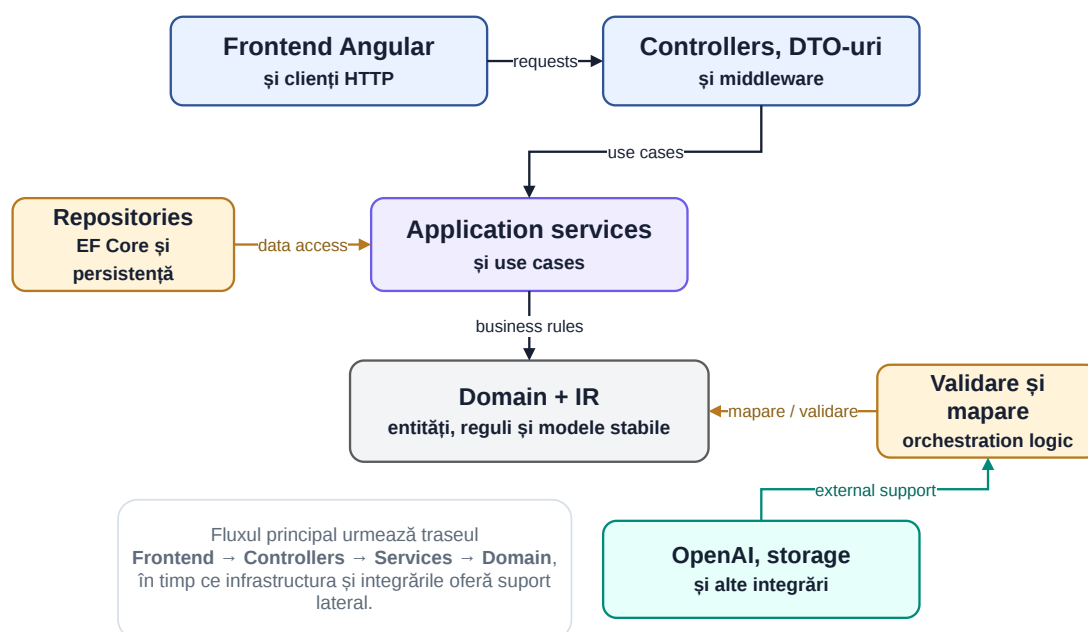


1. controller-ul primește cererea HTTP și verifică aspectele de intrare;
2. serviciul de aplicație aplică logica de business;
3. repository-ul sau adaptorul extern execută persistența sau integrarea necesară;
4. rezultatul este mapat și întors către client.

Această structură reduce riscul ca un controller să devină prea încărcat. De exemplu, un endpoint din 'ProjectsController' nu decide singur cum se construiește un slug, cum se șterg asset-urile sau cum se tratează un fork. Toate acestea sunt delegate către 'ProjectService', care la rândul lui colaborează cu repository-ul și cu storage-ul pentru assets.

Avantajul acestei structuri este vizibil și în testare. Pentru că serviciile sunt relativ bine delimitate, ele pot fi verificate independent de transportul HTTP. De aceea, în proiect există teste directe pentru 'AuthService', 'ProjectService', 'UserService' și 'ConverterEngine', nu doar teste la nivel de controller.

Aceeași organizare poate fi interpretată și printr-o lentilă apropiată de *clean architecture*. Regulile esențiale ale produsului și reprezentările interne trebuie să rămână în centru, iar infrastructura, transportul HTTP și integrarea cu servicii externe trebuie să depindă de acest nucleu, nu invers. Figura 4.3 sintetizează această idee pentru backend-ul Favigon.



**Figura 4.3:** Vedere de tip clean architecture pentru backend-ul Favigon

## 4.8 Organizarea frontend-ului pe feature-uri

Frontend-ul este organizat în jurul conceptelor 'core', 'shared' și 'features'. 'core' conține modelele, serviciile API, interceptorii și utilitarele globale. 'shared' conține componente reutilizabile. 'features' grupează zonele funcționale ale aplicației: autentificare, canvas, explore, profil, settings și stars.

Această structură este potrivită pentru proiect deoarece cea mai mare complexitate este concentrată într-un singur feature, anume 'canvas'. În loc să împrăștii logica editorului în toată aplicația, serviciile, mapper-ele și utilitarele specifice au fost păstrate aproape de acest feature. În practică, această alegere a redus numărul de dependențe surpriză și a făcut fluxul de lucru mai ușor de urmărit.

## 4.9 Modelul de date conceptual

Din punct de vedere conceptual, modelul de date are câteva entități centrale: utilizatorul, proiectul, conturile externe legate de utilizator, relațiile de follow dintre utilizatori și relațiile de like sau bookmark dintre utilizatori și proiecte. Designul efectiv al unui proiect este stocat ca JSON în cadrul proiectului, deoarece structura lui este prea flexibilă pentru o modelare exclusiv relațională. Vederea conceptuală simplificată apare în Figura 4.4.

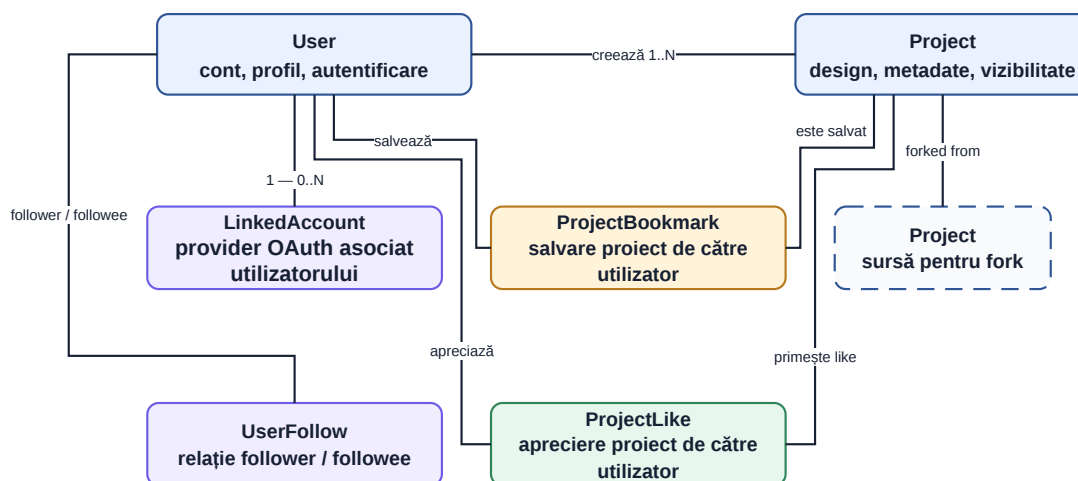


Figura 4.4: Model conceptual simplificat al datelor principale

Din cod se observă și câteva decizii importante. Proiectul are câmpuri pentru 'Slug', 'IsPublic', 'ThumbnailDataUrl', 'ViewCount' și 'ForkedFromProjectId'. Utilizatorul are câmpuri pentru 'HasPassword', 'IsTwoFactorEnabled', datele de resetare a parolei și date temporare pentru fluxul 2FA. Aceste câmpuri arată că modelul este orientat clar spre un produs complet, nu doar spre stocarea unui desen.

Pentru a trece de la nivelul de domeniu la nivelul de implementare, Figura 4.5 prezintă un ERD simplificat extras din configurația Entity Framework Core. El evidențiază entitățile persistente, cheile principale și relațiile care merită discutate la nivel de licență.

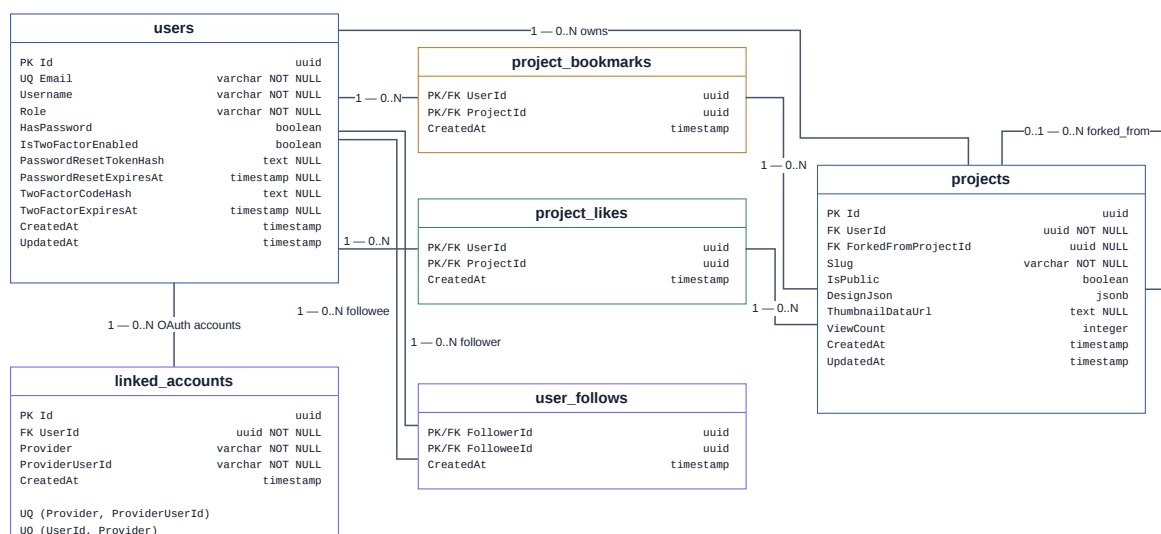


Figura 4.5: ERD simplificat pentru schema fizică a bazei de date

ERD-ul scoate în evidență și câteva constrângeri importante din implementare: unicitatea email-ului pentru utilizatori, unicitatea conturilor externe pe perechile (Provider, ProviderUserId) și (UserId, Provider), cheile compuse pentru relațiile sociale și autoreferința ForkedFromProjectId din projects. Astfel, schema nu stochează doar resurse izolate, ci și legăturile necesare pentru reutilizare, explorare și trasabilitatea fork-urilor.

Această structură arată explicit și compromisul de modelare adoptat în proiect. Datele cu structură stabilă, cum sunt identitatea utilizatorului, relațiile sociale și metadatele proiectelor, rămân relaționale. În schimb, documentul efectiv al interfeței este păstrat compact în DesignJson, deoarece arborele de elemente, stiluri și layout-uri este prea flexibil pentru a fi mapat eficient în tabele relaționale fine. În plus, timestamp-urile CreatedAt și UpdatedAt sunt menținute automat în contextul de persistență, ceea ce reduce riscul de inconsistență între creare, actualizare și export ulterior.

## **4.10 Fluxurile principale**

### **4.10.1 Fluxul de autentificare**

Fluxul de autentificare începe cu un request către 'AccountController', este procesat în 'AuthService', iar la final backend-ul emite cookie-urile necesare. Dacă utilizatorul are 2FA activ, autentificarea este împărțită în două etape: validarea credențialelor și validarea codului temporar. Această separare este reflectată și în cod, prin token-ul temporar specific fluxului 2FA.

### **4.10.2 Fluxul de editare**

La deschiderea unui proiect, frontend-ul încarcă datele proiectului și apoi designul asociat. 'CanvasPersistenceService' coordonează persistarea automată și sincronizarea thumbnail-ului, iar starea editorului este împărțită între mai multe servicii specializate. Din punct de vedere al utilizatorului, totul ar trebui să pară unitar. Din punct de vedere intern, însă, fluxul este descompus în pași mai mici tocmai pentru a păstra controlul asupra lui.

### **4.10.3 Fluxul de export**

Pentru export, editorul produce reprezentarea intermediară, apoi o trimite către backend prin 'ConverterController'. 'ConverterService' validează structura și apelează 'ConverterEngine', care generează fișierele pentru framework-ul ales. Acest flux este important deoarece justifică separarea convertorului într-un proiect distinct.

### **4.10.4 Fluxul AI**

Pentru generare, frontend-ul trimite un prompt către 'AiDesignController'. Backend-ul poate răspunde fie clasic, fie prin streaming SSE. În spate, 'AiPipelineService' parcurge fazele de intenție, structură și stil, iar rezultatul final este transformat în date pe care frontend-ul le poate integra în editor.

## **4.11 Decizii de proiectare care au influențat cel mai mult implementarea**

Privind retrospectiv, există câteva decizii de proiectare care au avut impact major asupra întregului proiect.

Prima decizie a fost separarea clară între canvas și codul final exportat. O abordare bazată pe editarea directă a HTML-ului sau a DOM-ului ar fi produs probabil un sistem mai fragil și mai greu de extins.

A doua decizie a fost tratarea AI-ului ca pe un subsistem controlabil, nu ca pe o cutie neagră. De aici rezultă atât folosirea JSON Schema, cât și împărțirea pipeline-ului.

A treia decizie a fost păstrarea logicii editorului în servicii specializate. Fără acest pas, componenta principală de canvas ar fi devenit foarte repede imposibil de întreținut.

A patra decizie a fost folosirea unei baze relaționale pentru metadate și a unui câmp JSON pentru design. Deși este un compromis, el s-a dovedit eficient pentru tipul de informații gestionate.

## **4.12 Concluzii intermediare**

În urma analizei și proiectării, sistemul rezultat are o formă coerentă atât la nivel de produs, cât și la nivel tehnic. Cerințele au fost traduse într-o arhitectură cu straturi clare, actori bine definiți, fluxuri importante și un model de date adaptat problemei.

Capitolul următor trece de la această vedere de ansamblu la implementarea concretă a platformei: autentificare, proiecte, funcții sociale și editorul canvas.

## Capitolul 5

# Implementarea platformei Favigon

### 5.1 Privire de ansamblu asupra implementării

După etapa de proiectare, implementarea efectivă a platformei a urmărit aceeași idee centrală: fiecare zonă importantă a produsului trebuie să aibă responsabilități clare și puncte de integrare ușor de urmărit. În practică, asta a însemnat că nu a fost dezvoltat întâi un editor izolat și abia apoi restul produsului, ci aplicația a evoluat ca întreg.

Mai exact, partea de autentificare, management de proiecte și persistare a fost dezvoltată relativ devreme, pentru că fără ele editorul ar fi rămas un demo local. Apoi au fost consolidate editorul canvas și suportul pentru export. Într-o etapă ulterioară au fost integrate fluxurile AI și a fost extinsă componenta socială, astfel încât proiectele să poată fi nu doar create, ci și publicate, descoperite și reutilizate.

Rezultatul este o aplicație în care modulele sunt diferite ca rol, dar strâns legate între ele. De exemplu, editorul depinde de proiecte, proiectele depind de autentificare, generarea AI depinde de editor și de modelul intermediar, iar funcțiile sociale depind de modelul relațional al utilizatorilor și proiectelor.

### 5.2 Rutare și pornirea aplicației frontend

Configurația frontend-ului este definită în `ApplicationConfig`, prin `provideRouter`, `provideHttpClient` cu interceptori și `provideAnimations`. Alegerea este în linie cu stilul modern Angular bazat pe componente standalone. Din punct de vedere practic, ea reduce nevoia unor module centrale voluminoase și face mai clară relația dintre rutare, infrastructura HTTP și comportamentul global al aplicației.

Un alt detaliu util este activarea `provideZoneChangeDetection` cu `eventCoalescing`. Pentru un produs cu editor vizual, optimizarea este relevantă deoarece reduce numărul de actualizări declanșate de evenimente succesive și ajută la păstrarea unei interfețe mai fluide în scenarii de drag, selecție sau input repetat.

Rutarea este organizată astfel încât paginile principale să fie încărcate lazy și protejate diferențiat. Ruta rădăcină nu trimite utilizatorul într-o pagină statică, ci încearcă să încarce utilizatorul curent și redirecționează dinamic fie spre profilul acestuia, fie spre `/login`. În plus, editorul canvas folosește un `canDeactivate guard` care forțează flush-ul persistenței înainte de părăsirea paginii, ceea ce leagă direct infrastructura de navigare de cerința de robustețe a autosave-ului.

**Tabela 5.1:** Rute frontend principale și rolul lor în aplicație

| Rută                    | Scop                    | Observații de acces / navigare                    |
|-------------------------|-------------------------|---------------------------------------------------|
| /login                  | autentificare în sistem | loginEntryGuard; evită revenirea inutilă la login |
| /reset-password         | recuperare acces        | rută publică pentru resetarea parolei             |
| /project/:slug          | editorul canvas         | authGuard și canDeactivate pentru flush           |
| /project/:slug/pre view | previzualizare HTML     | necesită autentificare și randare separată        |
| /explore                | descoperire de proiecte | punct de intrare în zona socială                  |
| /settings               | configurări de profil   | disponibilă doar utilizatorului autentificat      |
| /stars                  | proiecte salvate        | afișează bookmark-urile proprii                   |
| /:username              | profil public / propriu | destinație implicită după încărcarea sesiunii     |

Interacțiunea dintre rutare și infrastructura HTTP este completată de doi interceptori importanți: unul pentru trimiterea credențialelor către API și altul pentru încercarea automată de refresh atunci când sesiunea curentă expiră. Din punct de vedere al utilizatorului, această logică face ca aplicația să pară mai continuă și mai coerentă; din punct de vedere tehnic, ea mută comportamentele repetitive într-un strat transversal, în loc să fie duplicate în fiecare pagină sau serviciu.

## 5.3 Modulul de autentificare și profil

### 5.3.1 Fluxurile de bază

Modulul de autentificare este implementat în jurul ‘AccountController’ și al serviciului ‘AuthService’. La nivel de API există endpoint-uri pentru register, login, logout, refresh, resetare parolă, setare parolă, schimbare parolă, autentificare prin GitHub și Google, precum și pentru activarea sau dezactivarea autentificării în doi pași.

Pe partea de backend, ‘AuthService’ face mai mult decât o simplă verificare de credențiale. El normalizează datele de intrare, verifică unicitatea utilizatorului, gestionează hash-uirea parolei, tratează fluxurile OAuth, construiește token-urile JWT și emite un token temporar pentru etapa intermediară a autentificării în doi pași. În plus, aceeași zonă gestionează și resetarea parolei prin token transmis prin email.

Din punct de vedere al produsului, autentificarea nu este tratată ca o singură operație de tip “email + parolă”. În practică, utilizatorii pot intra în sistem prin mai multe căi și pot avea conturi legate la furnizori diferiți. Din acest motiv există și entitatea ‘LinkedAccount’, folosită pentru asocierea unui utilizator local cu identități externe.

### 5.3.2 JWT în cookie și refresh token

O decizie importantă a fost transportul token-ului de acces prin cookie HTTP only. În implementare, controller-ul nu întoarce token-ul către frontend pentru stocare manuală în ‘localStorage’, ci îl plasează în cookie-ul ‘jwt’. Refresh token-ul este păstrat separat, într-un cookie dedicat cu path limitat la endpoint-ul de refresh.

Această alegere simplifică partea de client și reduce expunerea unor date sensibile în codul din browser. În același timp, introduce și obligația de a trata atent expirațiile și reînnoirea sesiunii. De aceea, în frontend există și interceptorul de refresh, care încearcă revalidarea sesiunii când întâlnește un răspuns care indică expirarea token-ului de acces.

### 5.3.3 Autentificarea în doi pași

În cadrul proiectului, 2FA este implementat prin cod temporar trimis prin email. La prima vedere, acesta nu este cel mai avansat model posibil, dar pentru scopul aplicației este o soluție rezonabilă și ușor de demonstrat. În plus, implementarea este structurată destul de curat: există flux separat pentru activare, dezactivare și verificare la login, iar token-ul temporar pentru sesiunea de autentificare incompletă este diferit de token-ul normal de acces.

Această funcționalitate nu rămâne la nivel de extensie izolată. Ea este integrată atât în serviciul de autentificare, cât și în controller, în opțiuni și în fluxul de UI, ceea ce îi conferă rolul unei măsuri reale de securitate operațională.

### 5.3.4 Profilul utilizatorului

Pe lângă autentificare, utilizatorul își poate administra profilul: nume afișat, bio, imagine și legături OAuth. ‘UsersController’ expune endpoint-uri pentru profilul curent, încărcarea imaginii de profil, căutarea de utilizatori și vizualizarea profilurilor publice. În plus, există endpoint-uri pentru follow și unfollow, precum și pentru listele de followers și following.

Această parte a aplicației are un rol dublu. Pe de o parte este o funcționalitate de produs normală. Pe de altă parte, ea oferă baza pentru zona socială și pentru ideea că proiectele au autori, pot fi urmărite și pot circula între utilizatori.

## 5.4 Gestionarea proiectelor

### 5.4.1 Ciclul de viață al proiectelor

‘ProjectService’ este una dintre cele mai importante piese din backend, deoarece concentrează logica legată de proiecte. El gestionează crearea proiectului, actualizarea metadatelor, ștergerea, salvarea designului, salvarea thumbnail-ului, upload-ul de imagini, bookmark-urile, like-urile și operația de fork.

La creare, proiectul primește un slug unic derivat din nume. La modificare, dacă numele se schimbă, slug-ul este recalculat. La ștergere, serviciul se ocupă și de curățarea asset-urilor asociate. Acest detaliu este important deoarece evită acumularea de fișiere orfane și arată că proiectul nu tratează persistența doar la nivel de bază de date, ci și la nivelul fișierelor încărcate.

### 5.4.2 Persistarea designului

Pentru design-ul propriu-zis al unui proiect, backend-ul păstrează JSON-ul serializat în câmpul ‘DesignJson’ al entității ‘Project’. Pe frontend, ‘CanvasPersistenceService’ coordonează încărcarea, transformarea și salvarea acestui document.

Un aspect important este că salvarea nu este exclusiv manuală. Editorul folosește persistare automată cu debounce și mecanisme de flush atunci când utilizatorul părăsește pagina sau proiectul trebuie sincronizat imediat. Acest comportament este important pentru UX. Dacă utilizatorul lucrează într-un editor de design și trebuie să apese explicit Save după fiecare pas important, experiența devine artificială și fragilă.

În implementare, persistarea automată este tratată explicit prin parametri observabili în cod: schimbările relevante programează o salvare după 500 ms, flush-ul așteaptă cel mult 4 s finalizarea persistenței, iar la erori temporare se folosesc până la 4 reîncercări cu backoff

exponențial pornind de la 2 s. Serviciul gestionează separat stările de *isSaving*, persistarea la ieșirea din pagină și generarea de thumbnail-uri după perioade scurte de inactivitate. Aceste detalii transformă editorul dintr-un demo interactiv într-un subsistem care poate susține utilizare repetată.

### 5.4.3 Thumbnail și asset-uri

Fiecare proiect poate avea și thumbnail. În editor, thumbnail-ul este generat automat pornind din canvas, inclusiv printr-o variantă bazată pe 'html2canvas'. Pe backend, fișierul este salvat separat, iar la update se invalidează vechiul 'ThumbnailDataUrl' dacă este cazul.

În plus, proiectul suportă upload de imagini pentru asset-uri folosite în design. Validarea fișierelor se face în backend, iar stocarea este abstractizată printr-un serviciu dedicat. Această decizie a fost utilă pentru că a evitat legarea prea strânsă a logicii de proiecte de detaliile concrete ale fișierelor.

Un detaliu important, mai puțin vizibil în interfață, este ciclul de viață al asset-urilor. La resalvarea designului, backend-ul compară asset-urile prezente în documentul nou cu cele din versiunea anterioară și poate elimina resursele care au devenit orfane. În mod similar, la ștergerea unui proiect sau a unui cont, sunt curățate atât asset-urile proiectelor, cât și imaginea de profil a utilizatorului. Această zonă nu schimbă experiența vizuală în mod direct, dar este importantă pentru a arăta că aplicația tratează responsabil și consistența stocării de fișiere.

## 5.5 Funcționalități sociale și explorare

Componenta socială este construită peste fluxurile de explorare, proiecte și utilizatori, împreună cu serviciile și repository-urile aferente. În această zonă, utilizatorii pot vedea proiecte populare sau recomandate, pot descoperi alți utilizatori, pot aprecia proiecte, le pot salva și le pot reutiliza prin fork.

Deși această zonă nu este nucleul tehnic al lucrării, ea schimbă semnificativ caracterul produsului. Fără ea, Favigon ar fi rămas un editor personal. Cu ea, proiectul capătă logică de comunitate: există autori, conținut public, indicatori de interes și relații între utilizatori.

În implementare, 'ExploreService' tratează separat trei fluxuri: proiecte *trending*, proiecte recomandate și utilizatori sugerați. Pentru proiectele populare este folosit un set limitat de rezultate, potrivit pentru afișarea rapidă a conținutului public. Pentru recomandări, backend-ul verifică mai întâi dacă utilizatorul autentificat urmărește deja alte conturi; dacă da, încearcă să construiască un răspuns personalizat, iar dacă nu există bază suficientă sau utilizatorul este anonim, revine la o selecție generală de proiecte recente și relevante. Pentru sugestiile de persoane, serviciul returnează informații derivate precum numărul de urmăritori, numărul de proiecte publice și starea *isFollowedByCurrentUser*. Această logică oferă zonei 'explore' un comportament mai apropiat de un produs real decât de o simplă listă statică.

Operația de fork este deosebit de interesantă deoarece leagă partea socială de partea de producție. Când un utilizator face fork la un proiect public, sistemul creează un nou proiect privat pentru el, cu design-ul copiat și cu legătura către proiectul-sursă. Astfel, reutilizarea nu este doar conceptuală, ci concretă și trasabilă.



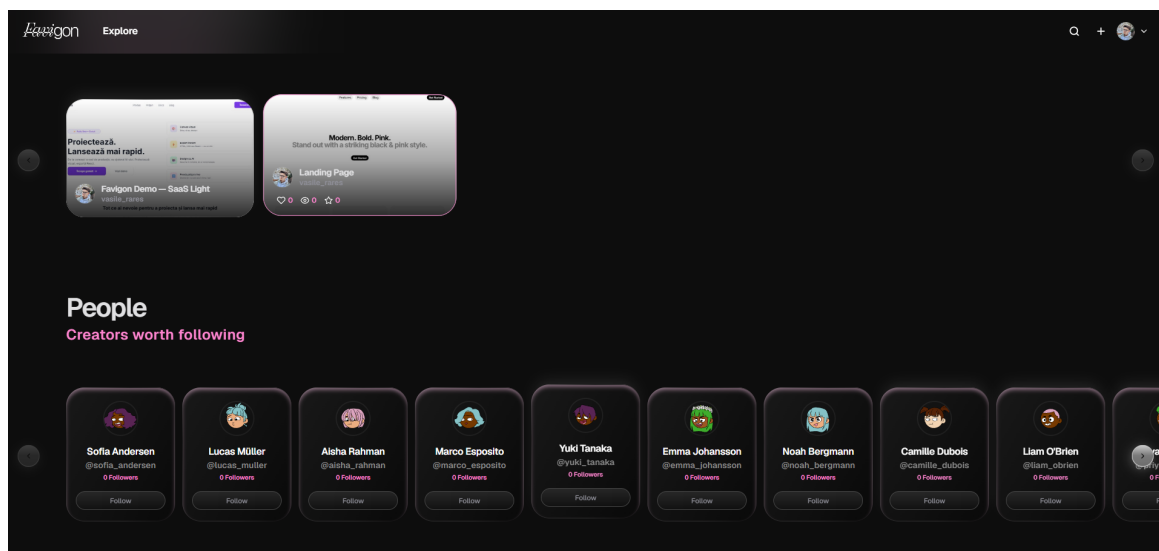


Figura 5.1: Interfața paginii Explore, cu proiecte publice și utilizatori sugerați

## 5.6 Suprafața API și distribuția responsabilităților

Din perspectiva backend-ului, aplicația expune o suprafață REST suficient de largă încât să merite descrisă separat. În forma actuală, API-ul este împărțit în șase controllere principale. Unele acoperă autentificarea și profilul, altele gestionează proiectele și componenta socială, iar cele rămase tratează generarea asistată de AI și exportul în cod.

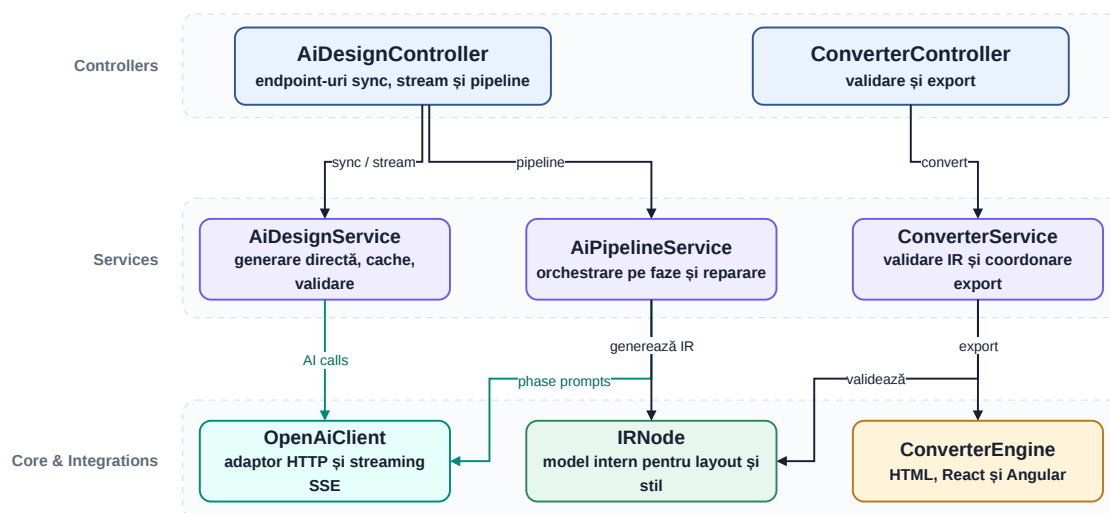
Distribuția responsabilităților pe controllere are un rol important în claritatea implementării. Zonele costisitoare sau sensibile, precum AI-ul și convertorul, sunt separate de ciclul de viață al proiectelor. În același timp, controllerele de produs, cum sunt cele pentru proiecte și utilizatori, pot rămâne focalizate pe resursele pe care le gestionează. Astfel, API-ul nu devine o colecție arbitrară de endpoint-uri, ci o hartă destul de coerentă a capabilităților aplicației.

Tabela 5.2: Controllere API și responsabilitățile lor dominante

| Controller          | Resurse / operații principale                      | Observații utile                                                 |
|---------------------|----------------------------------------------------|------------------------------------------------------------------|
| AccountController   | autentificare, sesiune, parole, OAuth, 2FA         | concentrează logica de intrare în sistem                         |
| ProjectsController  | CRUD proiecte, design, asset-uri, reacții, fork    | controllerul cel mai bogat din zona de produs                    |
| UsersController     | profil curent/public, follow, linked accounts      | leagă identitatea utilizatorului de zona socială                 |
| ExploreController   | trending, recomandări și sugestii de utilizatori   | pune în valoare date agregate și filtrate, nu doar resurse brute |
| AiDesignController  | generare directă, streaming, pipeline pe faze      | include atât răspuns sincron, cât și SSE incremental             |
| ConverterController | validare IR, export HTML/React/Angular, multi-page | separă clar validarea de generarea finală                        |

În implementare se observă și diferențierea infrastructurală a acestor zone. Controllerul AI este protejat atât prin autorizare, cât și prin politica dedicată de rate limiting ai. Controllerul de conversie are propria politică converter, iar ProjectsController aplică și limite diferite de dimensiune pentru upload de imagini, salvare design, flush și thumbnail. Aceste detalii arată că separarea controllerelor nu este doar conceptuală, ci are și efecte practice asupra securității, costului și fiabilității API-ului.

Dacă diagrama de arhitectură din capitolul precedent arată straturile mari, Figura 5.2 coboară un nivel și sintetizează colaborarea dintre clasele și serviciile dominante din fluxurile AI și export. Nu este o diagramă exhaustivă a întregului backend, ci o vedere structurală orientată pe subsistemele cele mai relevante pentru contribuția tehnică a lucrării.



**Figura 5.2:** Vedere structurală simplificată a principalelor clase și servicii backend

## 5.7 Editorul canvas

### 5.7.1 Rolul editorului în întregul produs

Editorul canvas este zona cea mai complexă din frontend și, în mod realist, centrul aplicației. În jurul lui gravitează generarea AI, persistarea, exportul și o mare parte din interacțiunile vizuale ale utilizatorului. În implementare, pagina principală a editorului ('CanvasPage') este o componentă mare, dar ea funcționează mai mult ca punct de orchestrare decât ca loc în care se află toată logica.

Practic, componenta injectează o suită întreagă de servicii: stare editor, viewport, istoric, clipboard, gesturi, context menu, persistare, generare, manager de pagini, stilizare DOM și altele. Separarea urmărește principiile interacțiunii de tip *direct manipulation*, unde utilizatorul operează pe obiecte vizibile, primește feedback imediat și poate reveni rapid asupra acțiunilor [29]. În același timp, păstrarea unui model intern separat de suprafața vizuală apropie editorul de direcțiile discutate în literatura despre combinarea editării directe cu reprezentări programabile [10]. Principalele servicii implicate sunt sintetizate în Figura 5.5.

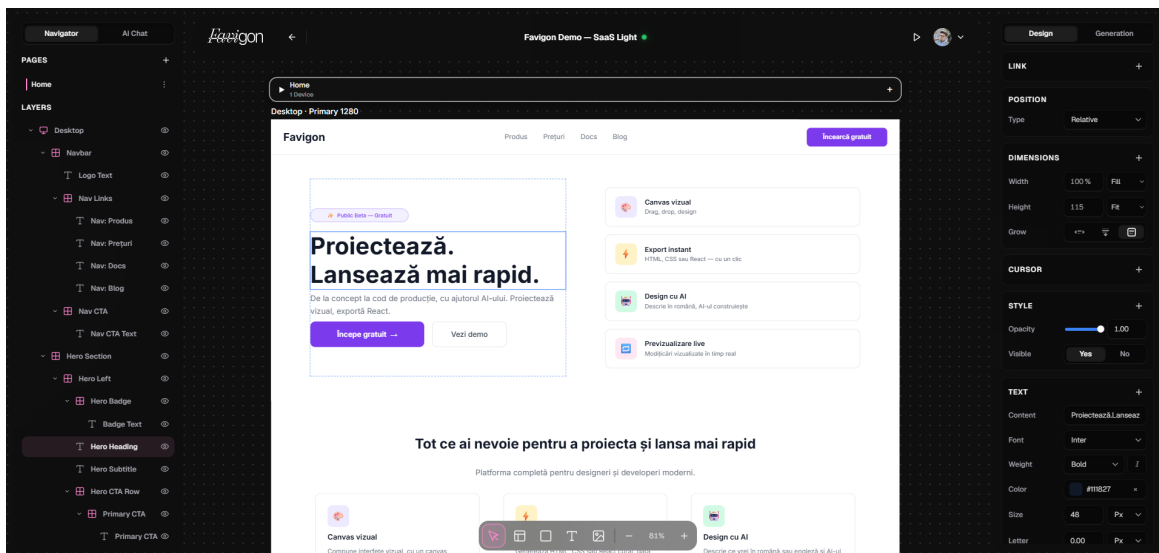


Figura 5.3: Interfața editorului canvas în timpul editării unui proiect

## 5.7.2 Starea editorului

‘CanvasEditorStateService’ păstrează semnalele principale ale editorului: paginile, pagina curentă, elementele selectate, unelele curente și alte stări derivate. Faptul că starea este separată într-un serviciu dedicat face posibilă partajarea ei între mai multe comportamente fără a transmite manual date prin multe nivele de componente.

Pe lângă starea explicită, editorul folosește și multe valori derivate: elementul curent, limitele selecției, lista de elemente vizibile, hărți auxiliare pentru copii, indexări rapide și stări temporare de drag sau resize. În cod se vede clar că aceste structuri sunt tratate ca parte din modelul intern al editorului, nu ca simple variabile locale.

## 5.7.3 Viewport, selecție și gesturi

Pentru ca editorul să fie util, utilizatorul trebuie să poată naviga în canvas, să selecteze elemente și să le manipuleze natural. ‘CanvasViewportService’ controlează zoom-ul, offset-ul și proprietățile CSS asociate. ‘CanvasGestureService’ gestionează drag, resize, rotate, editarea de text și calculul limitelor pentru overlay-uri.

O problemă practică interesantă a fost sincronizarea dintre modelul intern și limitele reale din DOM, mai ales pentru elemente transformate sau pentru copii din layout-uri flexibile. De aceea, în implementare există noțiuni precum *stable selection bounds*, cache pentru limite și mecanisme speciale pentru cazurile în care măsurarea live ar produce efecte vizuale nedorite. Este un exemplu de complexitate care nu apare în descrierea de nivel înalt, dar devine foarte concretă într-un editor bazat pe gesturi, selecție și feedback imediat.

## 5.7.4 Istoric și clipboard

‘CanvasHistoryService’ și ‘CanvasClipboardService’ susțin operații de tip undo, redo, copy și paste. Din punct de vedere al utilizatorului, acestea sunt funcții de bază. Din punct de vedere intern, ele sunt importante pentru că obligă sistemul să poată captura și reaplica stări coerente ale documentului.

Pentru istoric, soluția folosită este bazată pe snapshot-uri ale stării relevante. Nu este cea mai sofisticată variantă posibilă, dar pentru proiectul de față oferă un raport bun între simplitate și control. În plus, istoricul este suficient de clar încât să poată fi restaurat și după reîncărcarea proiectului.

### 5.7.5 Managementul paginilor

Editorul nu se limitează la o singură pagină. ‘CanvasPageManagerService’ și serviciile asociate permit existența mai multor pagini în același proiect, schimbarea paginii active și organizarea elementelor pe pagini. Această decizie influențează și exportul, deoarece face posibilă generarea multi-page în backend.

Din perspectiva lucrării, suportul multi-page este important deoarece mută Favigon din zona unui simplu constructor de ecrane izolate spre zona unui sistem care poate descrie un mic site sau o mică aplicație cu mai multe puncte de intrare.

### 5.7.6 Preview-ul interactiv

Pe lângă editorul propriu-zis, aplicația include și o pagină de preview separată (‘CanvasPreviewPage’), folosită atât de autorul proiectului, cât și de vizitatorii care accesează un proiect public. În loc să redea direct structura internă a canvas-ului, această pagină reconstruiește reprezentarea intermediară pentru pagina selectată și cere backend-ului generarea HTML/CSS. Rezultatul este injectat într-un ‘iframe’ prin ‘srcdoc’, ceea ce permite observarea produsului într-un mediu mai apropiat de randarea finală decât suprafața de editare.

Preview-ul nu este doar o captură statică. El permite schimbarea paginii curente, selectarea unui cadru de vizualizare, redimensionarea viewport-ului și navigarea între pagini prin link-uri interne, interceptate prin ‘postMessage’ între ‘iframe’ și aplicație. În plus, pentru proiectele publice, aceeași pagină expune acțiuni de tip like, star și fork. Din acest motiv, preview-ul funcționează și ca punte între editor, conversie și partea socială a produsului.

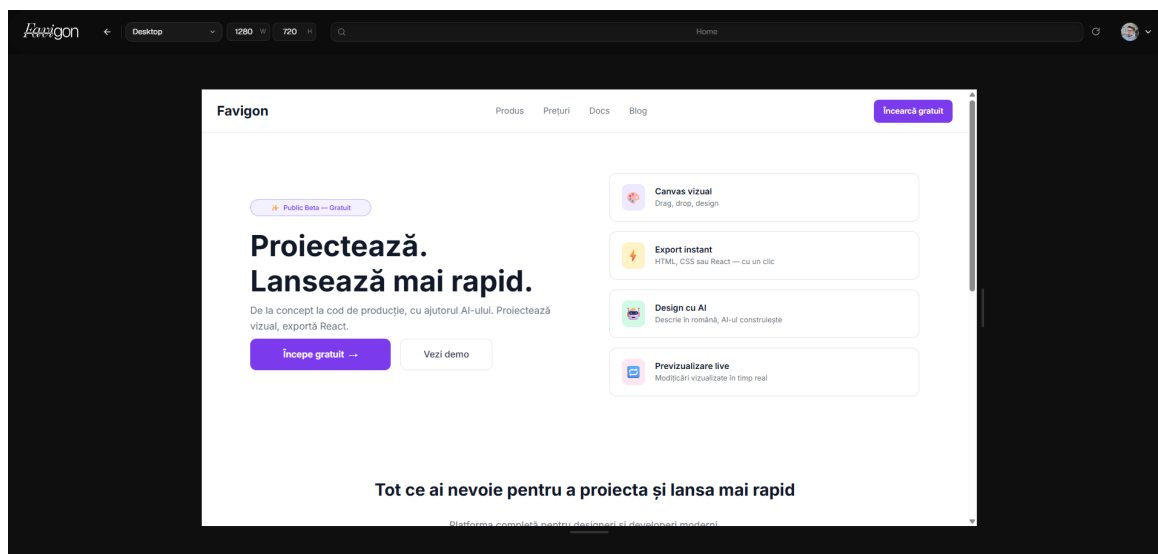


Figura 5.4: Pagina de preview folosită pentru vizualizarea rezultatului generat

### 5.7.7 Persistare și integrare cu restul aplicației

Editorul nu este un spațiu închis. El comunică permanent cu serviciile de proiect, cu zona de preview și cu motorul de conversie. ‘CanvasPersistenceService’ este legătura principală dintre starea editorului și backend. ‘CanvasGenerationService’ este puntea spre export. ‘CanvasAiChatPersistenceService’ și istoricul extind editorul dincolo de o simplă editare de suprafață.

Din punct de vedere practic, această integrare a fost mai grea decât scrierea unui editor minimal. Nu era suficientă mutarea unui element pe ecran; era necesară și posibilitatea de salvare, validare, export și reîncărcare a rezultatului fără inconsistențe majore.

### 5.7.8 Persistența locală a conversației AI

O extensie utilă a editorului este păstrarea locală a conversației AI și a contextului asociat. ‘CanvasAiChatPersistenceService’ folosește ‘IndexedDB’ pentru a salva, per proiect, mesajele recente, promptul aflat în lucru, modelul selectat și tab-ul activ al interfeței. Persistarea este făcută cu debounce de 300 ms, iar la restaurare sunt acceptate doar intrările mai noi de 7 zile. În plus, serviciul limitează mesajele păstrate la 40 și normalizează lungimea textelor, pentru a evita acumularea necontrolată de date în browser.

Această decizie completează bine autosave-ul clasic al designului. Documentul proiectului rămâne în backend, pentru a putea fi sincronizat și exportat, în timp ce conversația AI și contextul ei imediat rămân locale, ceea ce reduce trafic inutil și permite reluarea rapidă a sesiunii după reîncărcarea editorului. Împreună cu restaurarea istoricului și cu flush-ul la ieșire din pagină, rezultă un comportament mult mai apropiat de un instrument de lucru continuu decât de un simplu demo de generare.

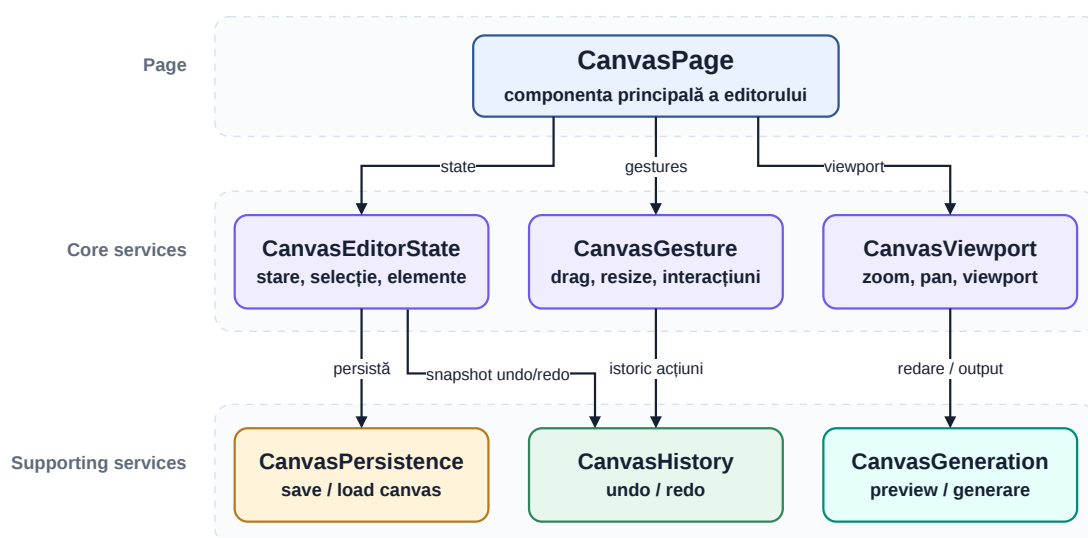


Figura 5.5: Servicii principale implicate în funcționarea editorului canvas

## 5.8 Decizii de implementare cu impact major

Privind la implementarea efectivă, există câteva decizii care au avut impact direct asupra stabilității și clarității codului.

Prima este faptul că logica specifică canvas-ului a fost păstrată în interiorul feature-ului ‘canvas’. În multe proiecte, utilitarele și serviciile migrează rapid spre zone generice, dar aici asta ar fi redus claritatea.

A doua este persistarea automată cu retry și flush controlat. Nu este o funcție spectaculoasă în demo, dar este foarte importantă în utilizare reală.

A treia este tratarea editorului ca pe o sumă de servicii coordonate, nu ca pe o singură componentă imensă care face totul. Deși ‘CanvasPage’ rămâne o componentă mare, mare parte din comportament este împinsă în servicii.

A patra este integrarea atentă dintre proiecte, thumbnail-uri și asset-uri. Fără această zonă, editorul ar fi avut o viață scurtă în afara unei sesiuni locale.

## 5.9 Concluzii intermediare

Implementarea platformei arată că Favigon este mai mult decât un proof of concept pentru generare AI. Autentificarea, proiectele, profilurile, funcțiile sociale și editorul canvas formează împreună baza produsului. În această bază, editorul este modulul cel mai complex, dar el nu ar avea aceeași valoare fără restul infrastructurii de produs.

Capitolul următor tratează componenta care diferențiază cel mai mult proiectul din punct de vedere tehnic: generarea asistată de AI și conversia interfețelor în cod.

## Capitolul 6

# Generarea asistată de AI și conversia interfețelor în cod

### 6.1 De ce nu este suficientă generarea directă

Una dintre ideile tentante atunci când lucrezi cu modele generative este să ceri direct rezultatul final: “generează-mi o pagină” sau chiar “generează-mi codul pentru această pagină”. Pentru demonstrații scurte, abordarea funcționează uneori surprinzător de bine. Totuși, pentru un produs software real, ea are câteva probleme serioase.

Prima problemă este controlul redus. Dacă modelul returnează direct cod, devine mai greu de separat unde s-a produs eroarea: în înțelegerea intenției, în structurarea layout-ului sau în alegerea stilurilor. A doua problemă este validarea. Codul poate arăta plauzibil fără să respecte constrângeri importante ale produsului. A treia problemă este reutilizarea. Dacă fiecare rezultat vine direct ca ieșire finală, nu există neapărat un nivel intern stabil pe care să îl poți edita și exporta în mai multe direcții.

Din aceste motive, în Favigon generarea asistată de AI este tratată ca un pipeline compus din etape distincte, nu ca pe un apel singular. În implementare, acest pipeline este orchestrat de ‘AiPipelineService’, iar integrarea cu modelul se face prin ‘OpenAiClient’.

### 6.2 Principiul pipeline-ului în trei faze

Soluția implementată în proiect împarte generarea în trei faze:

1. **intenție:** înțelegerea cererii și extragerea unei descrieri structurate a paginii;
2. **structură:** construirea arborelui intermediar de noduri, layout-uri și dimensiuni;
3. **stilizare:** aplicarea deciziilor vizuale pe structura deja generată.

Această împărțire pare simplă, dar în practică aduce mai multe avantaje. Fiecare etapă are un contract mai clar. Fiecare etapă poate fi testată sau inspectată separat. Fiecare etapă poate fi oprită anticipat dacă utilizatorul vrea doar o schiță. În plus, dacă rezultatul unei etape este invalid, eroarea este mai ușor de localizat. Fluxul general al acestei orchestrări este sintetizat în Figura 6.1.

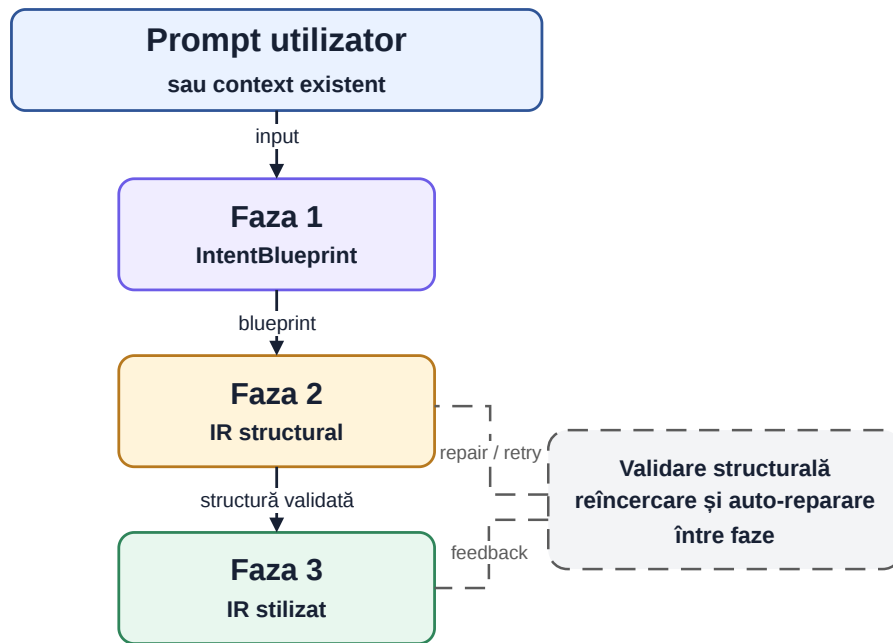


Figura 6.1: Pipeline-ul AI în trei faze folosit în Favigon

## 6.3 Faza 1: extragerea intenției

În prima etapă, sistemul nu încearcă să deseneze nimic și nu generează cod. Obiectivul este mai restrâns: să transforme promptul utilizatorului într-un *IntentBlueprint*. În implementare, acest blueprint conține informații precum tipul paginii, starea vizuală generală, personalitatea brandului, publicul țintă, CTA-ul principal și lista secțiunilor în ordinea lor.

Acest pas explicit este important din două motive. În primul rând, el forțează modelul să ia o decizie de structură înainte să intre în detalii estetice. În al doilea rând, oferă frontend-ului și backend-ului o reprezentare intermediară ușor de afișat, inspectat sau discutat.

În practică, această etapă seamănă mai mult cu o analiză de cerințe decât cu o generare vizuală. Promptul este reinterpretat într-un format intern. Dacă utilizatorul cere, de exemplu, o pagină de prezentare pentru un produs SaaS, rezultatul nu trebuie să fie imediat un hero cu gradient, ci o listă de secțiuni și intenții: navbar, hero, beneficii, social proof, CTA, footer și așa mai departe.

## 6.4 Faza 2: generarea structurii

A doua etapă este cea care construiește efectiv arborele intermediar de noduri și reprezintă etapa centrală a întregii abordări. În loc să fie cerute modelului HTML final sau componente de framework, sarcina transmisă este producerea unui arbore 'IRNode' cu layout, dimensiuni, padding, gap, poziționare și text.

Mai concret, structura generată include tipuri de noduri, copii, meta-informații, stiluri și reguli de layout. Promptul sistem pentru această fază este mult mai strict decât în prima etapă și include reguli explicite despre cum trebuie dimensionate secțiunile, cum trebuie folosite tipografiile, ce înseamnă 'fit-content', când se poate folosi '100'.

Această rigiditate este intenționată. Dacă în faza de intenție modelul are libertate mai mare, în faza structurală libertatea trebuie redusă. Altfel, ar exista prea multe variații greu de validat și greu de convertit.



## 6.5 Faza 3: stilizarea

A treia etapă pornește de la structura deja creată și aplică sistemul vizual: culori, gradient, umbre, raze, borduri, cursor și alte detalii de prezentare. În implementare, promptul pentru această fază conține o regulă foarte importantă: layout-ul și structura nu trebuie modificate. Altfel spus, stilizarea nu are voie să rescrie ceea ce a rezolvat deja etapa de structură.

Această separare între structură și stil este utilă atât pentru control, cât și pentru depanare. Dacă o pagină este logică, dar arată slab, știu că problema este în faza a treia. Dacă layout-ul este greșit, știu că problema este în faza a doua. În plus, această abordare seamănă mai bine cu modul în care lucrează și oamenii: întâi decid structura și ierarhia, apoi rafinează aspectul vizual.

## 6.6 Integrarea cu OpenAI API

În Favigon, comunicarea cu modelul este abstractizată prin 'OpenAIClient'. Acesta folosește endpoint-ul de *Chat Completions* și suportă atât răspuns obișnuit, cât și streaming [23]. Alegerea de a ascunde apelurile HTTP într-un adaptor de infrastructură păstrează serviciile de aplicație concentrate pe logică și face mai ușoară înlocuirea sau extinderea integrării în viitor.

Clientul construiește payload-ul cu mesaj de sistem, mesaj de utilizator, model configurabil și, unde este cazul, `response_format` bazat pe schemă JSON. În plus, pentru scenariile de streaming, clientul parcurge liniile SSE și extrage incremental conținutul transmis de model. În implementarea actuală, identificatorul modelului este citit din configurație, cu fallback la `gpt-5.4-mini`, iar cererile trimit atât `response_format`, cât și un prag explicit de `max_completion_tokens` pentru a limita comportamentul apelului la parametri observați și reproductibili.

Păstrarea identificatorului modelului în configurație, nu hardcodat în logică, este importantă deoarece partea experimentală a aplicației poate necesita ajustări de model în timp, iar aceste schimbări nu ar trebui să implice recompilarea serviciului de aplicație.

## 6.7 Structura cererilor AI și feedback incremental

Din punctul de vedere al API-ului, integrarea AI nu primește doar un simplu șir de text. Cererea de bază către endpoint-urile de design include promptul, un viewport țintă, modelul ales și, opțional, o structură IR existentă. Acest ultim câmp este important deoarece permite scenarii în care utilizatorul nu cere generarea unei pagini de la zero, ci rafinarea sau extinderea unei structuri deja existente. În cazul pipeline-ului pe trei faze, aceeași cerere mai include și parametrul `StopAfterPhase`, ceea ce face posibilă oprirea controlată după faza de intenție sau după faza structurală.

**Tabela 6.1:** Câmpuri relevante din cererile AI folosite de backend

| Câmp           | Semnificație                                  | Rol în procesare                                                           |
|----------------|-----------------------------------------------|----------------------------------------------------------------------------|
| Prompt         | descrierea textuală a interfeței dorite       | punctul de pornire pentru interpretarea intenției                          |
| ExistingIr     | structură intermediară existentă, opțională   | permite extindere, reparare sau rafinare în loc de generare complet nouă   |
| ViewportWidth  | lățimea de referință a paginii generate       | influențează presupunerile de layout și scară                              |
| Model          | identificatorul modelului folosit             | permite alegerea explicită între variantele configurate                    |
| StopAfterPhase | limită de oprire pentru pipeline-ul în 3 faze | utilă pentru schițe parțiale, depanare și observarea etapelor intermediare |

Suportul pentru streaming schimbă semnificativ și experiența utilizatorului. În loc să aștepte un singur răspuns final, frontend-ul poate primi evenimente incremental și poate afișa progresul procesării. În implementarea actuală, endpoint-urile de streaming trimit răspunsuri de tip `text/event-stream`, iar backend-ul emite linii SSE în care tipul evenimentului și conținutul sunt transmise separat. Pentru pipeline-ul în trei faze, evenimente precum `phase_start`, `phase_complete`, `result` sau `error` oferă o vizibilitate mult mai bună asupra procesului decât o simplă animație de încărcare.

**Listing 6.1:** Exemplu simplificat de evenimente SSE trimise în timpul pipeline-ului AI

```
event: phase_start
data: {"phase":1,"name":"intent"}

event: phase_complete
data: {"phase":1,"name":"intent"}

event: result
data: {"success":true,"stopAfterPhase":3}
```

Din punct de vedere operațional, fluxul complet dintre utilizator, frontend, controller, orchestration service și modelul AI este mai bine surprins printr-o diagramă de secvență. Figura 6.2 evidențiază atât schimburile principale de mesaje, cât și faptul că pipeline-ul nu este un singur apel, ci o succesiune controlată de faze cu feedback incremental.

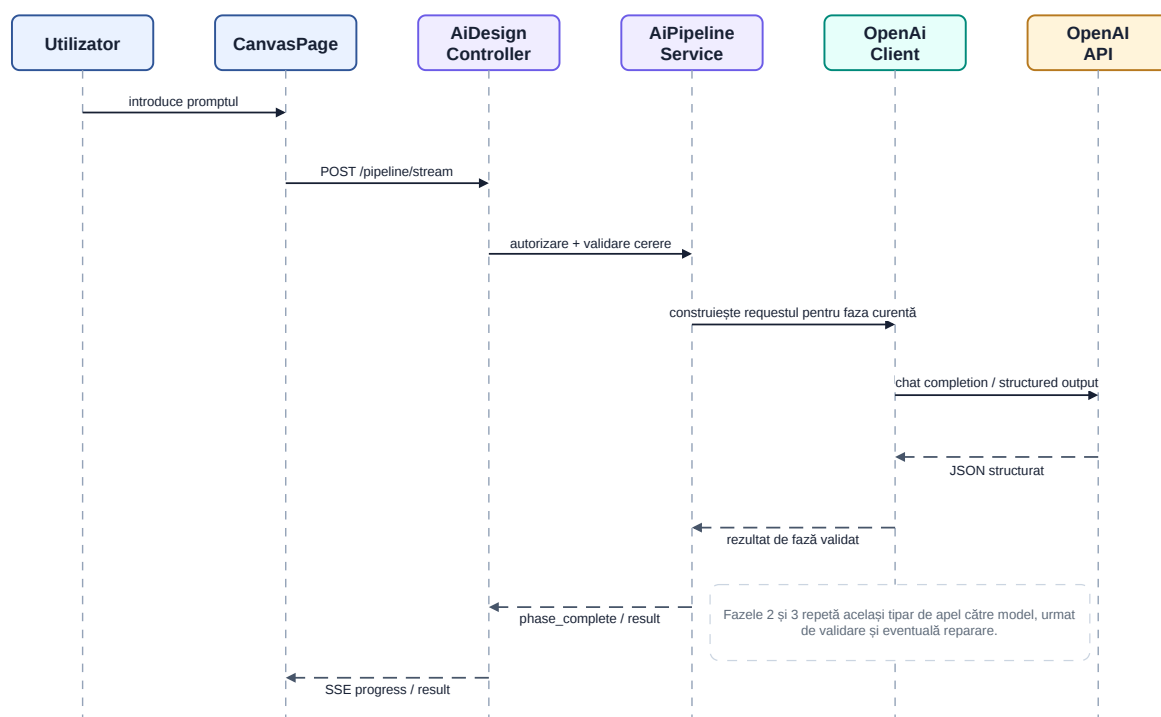


Figura 6.2: Diagrama de secvență simplificată pentru fluxul AI cu streaming

## 6.8 Rolul ieșirilor structurate

OpenAI documentează explicit posibilitatea obținerii unor ieșiri structurate, conforme cu un anumit format sau cu o anumită schemă [24]. În proiect, această capabilitate este foarte importantă. Dacă modelul ar întoarce doar text liber, parser-ul și validarea ar deveni mult mai fragile. În schimb, cu o schemă bine definită, aplicația poate detecta mai repede abaterile și poate decide dacă rezultatul este utilizabil sau nu.

Totuși, experiența din implementare arată că *structured output* nu este o soluție magică. El ajută la format, dar nu garantează automat calitatea semantică a rezultatului. De exemplu, un JSON perfect valid poate descrie o structură slabă sau o ierarhie de secțiuni nepotrivită. Din acest motiv, 'AiPipelineService' folosește atât deserializare și validare structurală, cât și reguli suplimentare și un pas de auto-reparare când rezultatul nu este acceptabil.

## 6.9 Mecanisme de validare și reparare

După fiecare etapă relevantă, rezultatul este analizat înainte să fie acceptat. Dacă în faza structurală arborele IR nu poate fi parsat sau nu respectă schema așteptată, serviciul încearcă un pas suplimentar de reparare. Acest pas trimite către model erorile de validare și o variantă trunchiată a rezultatului defect, cerând un JSON corectat.

Practic, modelul este tratat nu doar ca generator, ci și ca potențial asistent pentru repararea propriilor ieșiri. Această idee este utilă în contextul lucrării deoarece arată un mod mai realist de folosire a AI-ului: prima ieșire nu este presupusă perfectă, iar sistemul este proiectat explicit pentru a gestiona erori.

În plus, atât 'AiDesignService', cât și 'AiPipelineService' folosesc cache cu TTL de 10 minute pentru cereri identice și validează separat răspunsurile obținute. În varianta de pipeline există și posibilitatea de oprire după faza 1 sau 2, respectiv emiterea de evenimente SSE de tip

`phase_start` și `phase_complete`, ceea ce face procesul mai ușor de urmărit în interfață și în depanare.

## 6.10 Reprezentarea intermediară în detaliu

Structura 'IRNode' este elementul care leagă editorul, AI-ul și convertorul. Un nod include identificator, tip, proprietăți, layout, stil, poziționare, copii, variante responsive, metadate și, opțional, efecte de animație. Această structură este suficient de generală pentru a descrie atât containere, cât și text, imagini sau frame-uri de pagină.

În practică, modelul intermediar oferă trei beneficii majore.

Primul este **independența față de framework**. Un 'IRNode' nu este nici componentă Angular, nici JSX React, nici HTML final. De aceea poate fi transformat în toate aceste forme.

Al doilea este **validabilitatea**. Pentru că structura este explicită, poate fi verificată existența anumitor câmpuri, coerența layout-ului sau compatibilitatea dimensiunilor înainte de export.

Al treilea este **trasabilitatea**. Când ceva nu merge bine în export, structura intermediară poate fi inspectată și se poate vedea dacă problema vine din editor, din AI sau din convertor.

## 6.11 Maparea dintre canvas și IR

Pe partea de frontend, designul construit în editor nu este trimis direct către convertor în forma nativă a canvas-ului. În schimb, există mapper-e care transformă datele editorului în IR și invers. Acest detaliu este foarte important pentru arhitectura proiectului.

Dacă editorul ar fi legat direct de codul final, orice schimbare în convertor ar risca să rupă și logica din canvas. Prin introducerea mapper-elor, este creat un punct de decuplare. Editorul poate evolua într-o anumită direcție, iar convertorul în alta, atâta timp cât contractul IR rămâne coerent.

Maparea inversă, din IR în elemente de canvas, este la fel de utilă. Ea face posibilă integrarea rezultatelor generate de AI direct în editor, astfel încât utilizatorul să nu primească un export static, ci un design pe care îl poate modifica în continuare.

## 6.12 Exemplu concret de flux de la prompt la IR și export

Pentru a înțelege mai clar legătura dintre subsisteme, este util un exemplu simplificat. Presupunând un prompt de tipul „crează o pagină landing pentru o aplicație de task management, cu hero, beneficii și call-to-action”, prima fază nu produce HTML și nici noduri finale, ci o descriere structurată a intenției: tip de pagină, audiență, ton vizual și secțiuni în ordinea așteptată. A doua fază transformă această intenție într-un arbore IR cu frame principal, containere, text și butoane, iar a treia aplică stilurile. Abia după aceste etape structura ajunge la convertor.

În etapa de export, convertorul primește fie un singur IRNode, fie o listă de pagini. Pentru cazul simplu single-page, ConverterController apelează `validarea` și apoi `Generate`. Pentru cazul multi-page, aceeași rută de generare poate primi `Pages`, iar endpoint-ul dedicat pentru fișiere folosește `GenerateMultiPage`. Rezultatul nu este doar o bucată de cod brut, ci un set coerent de artefacte adaptate framework-ului cerut: HTML și CSS pentru varianta statică, respectiv fișiere de componentă, stil și structură pentru React sau Angular.

Acest exemplu arată de ce separarea pe etape este importantă. Dacă problema apare în secționarea paginii sau în ierarhia elementelor, ea poate fi localizată înainte de convertor. Dacă structura este corectă, dar codul rezultat nu respectă așteptările, zona de investigație se mută

în motorul de export. În practică, această trasabilitate reduce mult costul depanării și face sistemul mai explicabil în comparație cu o abordare directă „prompt în, cod afară”.

## 6.13 Motorul de conversie

### 6.13.1 Separarea în proiect dedicat

Motorul de conversie este implementat în proiectul ‘Favigon.Converter’. El este separat explicit de ‘Favigon.Application’ deoarece are propria logică de validare, transformare și emitere. Dacă ar fi rămas îngropat în același serviciu cu restul aplicației, ar fi fost mai greu de testat și mai greu de extins.

### 6.13.2 Validarea

Înainte de generare, ‘ConverterService’ cere motorului verificarea structurii IR. Dacă apar erori, ele sunt raportate și procesul se oprește. Această etapă este necesară pentru că ieșirea finală ar deveni altfel imprezvizibilă. Este preferabil refuzul unui export decât generarea de cod inconsistent fără semnalarea problemei.

### 6.13.3 Generarea single-page și multi-page

‘ConverterEngine’ suportă atât generarea pentru o singură pagină, cât și generarea pentru mai multe pagini. Pentru multi-page, motorul produce seturi de fișiere și gestionează numele de pagini, slug-urile și structura finală. În cazul exportului Angular sau React, el generează și fișiere de proiect minimal, rute și fișiere CSS.

Această capabilitate este importantă deoarece proiectul nu se reduce la o singură artboard. Odată ce editorul suportă mai multe pagini, exportul trebuie să poată păstra această structură.

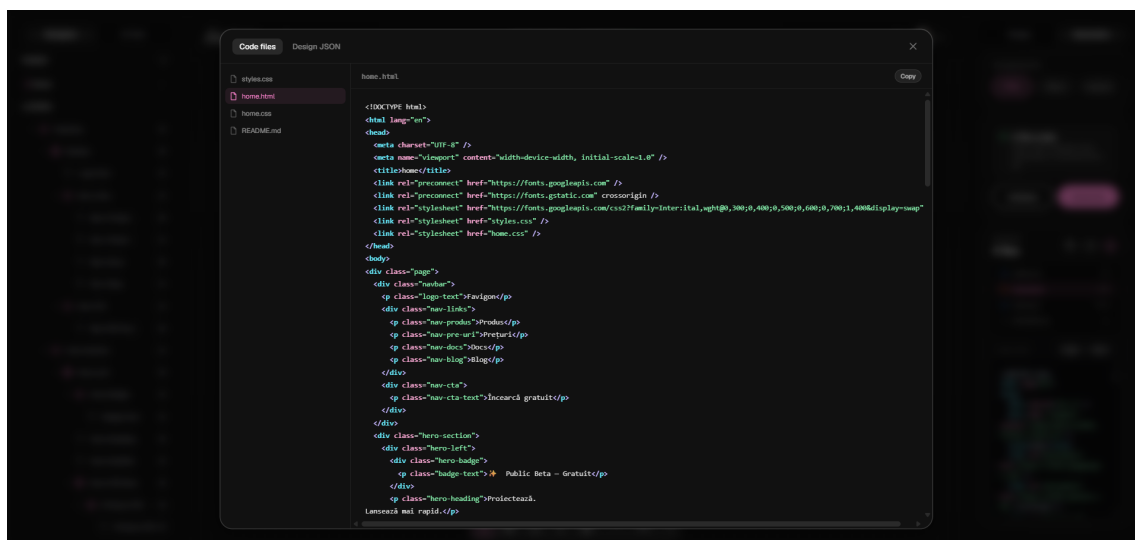


Figura 6.3: Exemplu de artefacte generate și inspectate în interfața de export

### 6.13.4 Responsive output

O componentă mai tehnică și mai interesantă este exportul responsive. În loc să dublez pur și simplu tot arborele pentru fiecare breakpoint, motorul produce artefactele principale și apoi diferențe CSS pentru breakpoints suplimentare. În testele existente se vede că sunt tratate cazuri precum noduri prezente doar pe anumite breakpoint-uri, schimbări de ordine pentru copii flex și diferențe de pseudo-clase sau keyframes.

Aceasta este una dintre funcțiile care arată cel mai clar că proiectul nu se oprește la un export superficial. Există o preocupare reală pentru cum este construit rezultatul final și pentru cât de mult poate fi controlat. Fluxul complet canvas → IR → artefacte de export este rezumat în Figura 6.4.

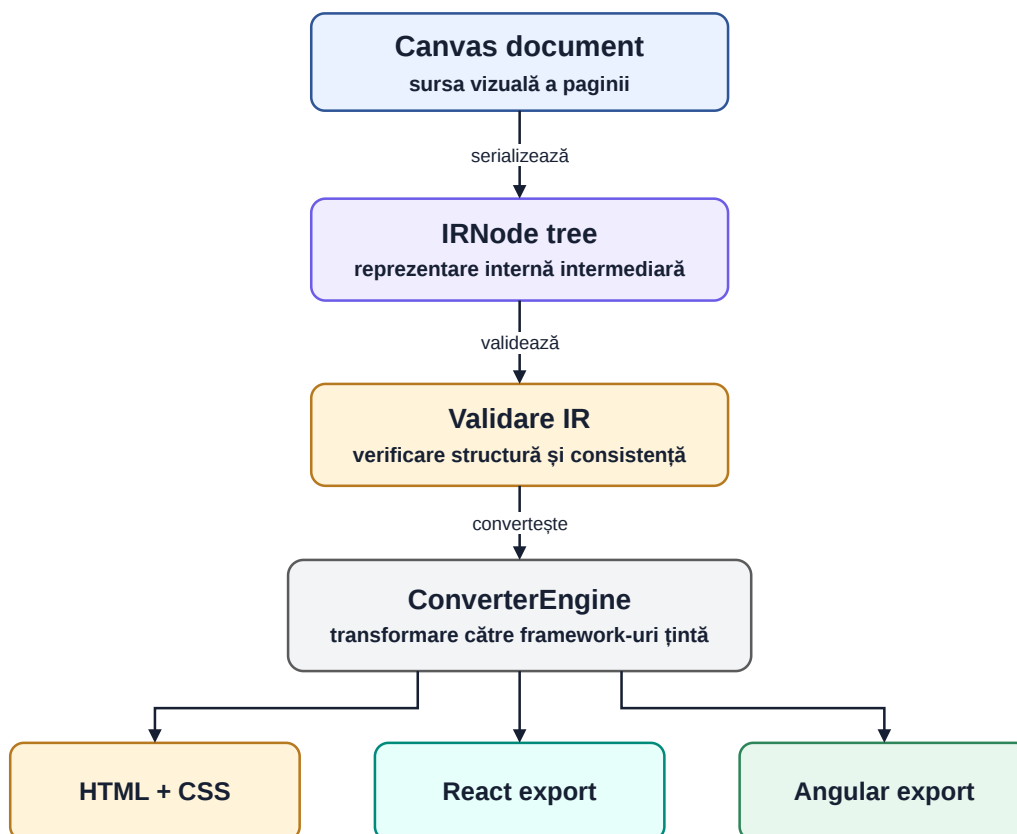


Figura 6.4: Fluxul design-to-code bazat pe reprezentarea intermediară

## 6.14 Legătura dintre AI și convertor

Un aspect foarte important este faptul că AI-ul și convertorul nu sunt două subsisteme complet separate. Ele comunică prin aceeași reprezentare intermediară. Fără această alegere, rezultatul generat de AI ar fi fost mult mai greu de transformat în cod într-un mod controlat.

Pe scurt, pipeline-ul AI generează un arbore care este suficient de bogat pentru a fi editat și suficient de formal pentru a fi convertit. În momentul în care această condiție este îndeplinită, editorul devine interfața de rafinare, iar convertorul devine ieșirea tehnică. Această relație dintre cele trei module — AI, editor și convertor — reprezintă contribuția tehnică principală a întregului proiect.

## 6.15 Limitări actuale

Deși soluția funcționează și este coerentă, există și limite.

În primul rând, calitatea rezultatului AI depinde încă mult de prompt și de regulile incluse în mesajele de sistem. Cu alte cuvinte, caracterul probabilistic al modelului nu este eliminat; el este doar controlat mai bine.

În al doilea rând, reprezentarea intermediară este suficient de bogată pentru scopurile actuale, dar nu acoperă toate situațiile pe care le-ar avea un constructor comercial matur.

De exemplu, colaborarea în timp real, constrângerile mai avansate de componentizare sau integrarea automată cu design systems externe ar necesita extinderi suplimentare.

În al treilea rând, exportul generat este orientat spre claritate și portabilitate, nu neapărat spre cele mai sofisticate convenții ale fiecărui ecosistem. Pentru o platformă academică, acest compromis este acceptabil.

## **6.16 Concluzii intermediare**

Capitolul de față descrie partea cea mai originală din proiect. Generarea asistată de AI nu este tratată ca un artificiu separat, ci ca parte a unei arhitecturi care include reprezentare intermediară, validare, reparare și conversie în cod. Tocmai această integrare controlată face diferența între o demonstrație izolată și un produs software coerent.

În capitolul următor sunt urmărite scenarii concrete de utilizare, pentru a observa cum colaborează în practică editorul, AI-ul, preview-ul, exportul și componenta socială. Abia după această etapă este discutată explicit validarea sistemului prin măsuri de securitate și testare.

## Capitolul 7

# Scenarii de utilizare și evaluare practică

### 7.1 Rolul acestui capitol în contextul lucrării

După prezentarea arhitecturii, a implementării și a mecanismelor de generare asistată de AI, este utilă o etapă intermediară în care aplicația să fie privită ca produs utilizabil, nu doar ca ansamblu de module. În practică, valoarea unui sistem precum Favigon nu rezultă numai din existența unui editor canvas, a unui pipeline AI sau a unui convertor, ci din felul în care acestea lucrează împreună într-un flux coerent pentru utilizator.

Din acest motiv, capitolul de față urmărește câteva scenarii reprezentative, alese astfel încât să acopere atât utilizarea clasică a editorului, cât și scenariile care implică generare asistată, publicare și reutilizare. Scopul nu este repetarea implementării descrise deja în capitolele anterioare, ci observarea modului în care subsistemele colaboratoare se comportă atunci când sunt folosite cap-coadă.

În plus, această abordare ajută și la justificarea deciziilor de proiectare. Unele alegeri care par excesiv de tehnice atunci când sunt privite izolat, de exemplu reprezentarea intermediară, persistarea automată sau separarea convertorului într-un proiect dedicat, devin mai ușor de înțeles când sunt raportate la pașii parcurși de utilizator într-un scenariu concret.

### 7.2 Criteriile urmărite în evaluarea practică

Scenariile alese nu urmăresc performanțe de benchmark și nici o comparație formală cu produse comerciale. Ele au un obiectiv mai simplu și mai potrivit pentru lucrarea de față: să arate dacă fluxurile importante ale produsului pot fi parcurse fără rupturi majore între editare, persistare, generare, preview, export și publicare.

La nivel concret, au fost urmărite patru criterii.

1. **coerența fluxului de lucru**, adică măsura în care utilizatorul poate trece natural de la o etapă la alta;
2. **trasabilitatea rezultatului**, adică posibilitatea de a înțelege de unde provine un anumit rezultat și în ce zonă apare o problemă;
3. **stabilitatea operațiilor frecvente**, precum salvarea, schimbarea paginilor, preview-ul și exportul;
4. **continuitatea dintre zona de creație și zona socială**, adică publicarea, explorarea și reutilizarea proiectelor.

Aceste criterii au fost alese deoarece reflectă direct obiectivele proiectului. Favigon nu își propune doar să deseneze elemente pe un canvas și nici doar să apeleze un model AI, ci să ofere un traseu relativ complet de la idee la rezultat reutilizabil. În consecință, o evaluare practică relevantă trebuie să urmărească exact această continuitate.

### 7.3 Scenariul 1: creare manuală și export al unui proiect

Primul scenariu urmărește utilizarea aplicației fără generare AI, pentru a observa în ce măsură editorul, persistarea și exportul pot susține singure fluxul principal de lucru. Scenariul pornește



de la crearea unui proiect nou și continuă cu editarea lui în canvas, verificarea prin preview și obținerea artefactelor finale de export.

În practică, utilizatorul începe prin crearea unui proiect și deschiderea editorului dedicat. Odată încărcată pagina, interfața principală a editorului oferă atât zona de lucru, cât și panourile laterale necesare pentru navigarea printre pagini, inspectarea structurii și modificarea proprietăților. Figura 5.3 ilustrează această stare de lucru și este relevantă deoarece arată că editorul nu este tratat ca o simplă suprafață grafică, ci ca un spațiu în care structura documentului și proprietățile elementelor rămân vizibile și controlabile.

În acest scenariu, utilizatorul construiește manual o pagină de prezentare simplă, formată dintr-un navbar, o secțiune hero, o zonă de beneficii și un call-to-action final. Din perspectiva experienței de lucru, un aspect important este faptul că operațiile de selecție, mutare, redimensionare și configurare a proprietăților sunt integrate într-un singur flux. Utilizatorul nu lucrează separat într-un “mod de design” și într-un “mod de structură”, ci vede în același timp atât rezultatul vizual, cât și organizarea internă a elementelor.

Această observație este importantă pentru că arată rolul serviciilor descrise în capitolul precedent. ‘CanvasEditorStateService’, ‘CanvasGestureService’ și ‘CanvasViewportService’ nu apar direct în interfață sub forma unor concepte teoretice, dar efectul lor cumulat devine vizibil tocmai în astfel de operații simple. Dacă ele nu ar colabora corect, editorul ar părea imediat inconsistent: selecția ar rămâne în urmă, gesturile ar reacționa prost sau starea documentului s-ar rupe de suprafața vizuală.

Un al doilea aspect urmărit în acest scenariu este persistarea. În timpul editării, modificările nu sunt gândite ca acțiuni care trebuie confirmate permanent de utilizator prin salvare manuală. În schimb, editorul folosește autosave cu debounce și flush controlat la ieșirea din pagină. Din punct de vedere practic, această alegere este foarte importantă. Într-un produs de tip design tool, orice dependență excesivă de salvare manuală ar rupe ritmul de lucru și ar transforma editorul într-un prototip fragil.

În cazul Favigon, comportamentul observat este apropiat de cel așteptat pentru un instrument de lucru continuu. Utilizatorul poate face mai multe modificări succesive, poate schimba selecția, poate naviga între pagini și poate reveni asupra unor elemente fără să aibă impresia că lucrează împotriva mecanismului de persistență. Acest lucru nu elimină toate riscurile, dar arată că partea de autosave nu a rămas un detaliu de implementare, ci susține direct experiența de utilizare.

După editare, scenariul continuă cu verificarea rezultatului în pagina de preview. Aici apare una dintre diferențele importante dintre suprafața de editare și produsul rezultat. În locul reproducerii directe a documentului intern al editorului, aplicația reconstruiește reprezentarea intermediară și cere backend-ului generarea HTML/CSS pentru pagina curentă. Figura 5.4 arată această etapă și evidențiază faptul că preview-ul funcționează ca un pas de validare realistă, nu ca o simplă captură a canvas-ului.

Din punct de vedere practic, preview-ul este util din două motive. Primul este validarea vizuală: utilizatorul poate observa mai ușor cum arată rezultatul într-un mediu mai apropiat de randarea finală. Al doilea este validarea fluxului tehnic: dacă designul arată corect în editor, dar nu se mai păstrează în preview, problema nu mai este în zona de manipulare vizuală, ci în maparea către IR sau în convertor.

Ultimul pas al scenariului este exportul. Odată ce pagina a fost verificată, utilizatorul poate cere generarea artefactelor finale. Figura 6.3 ilustrează o astfel de situație și arată că ieșirea nu constă într-un singur bloc de text, ci într-un set de fișiere inspectabile. Pentru lucrarea de față, acest detaliu este important deoarece arată că fluxul nu se oprește la o promisiune de tip “design-to-code”, ci duce efectiv la un rezultat care poate fi citit și reutilizat.

Per ansamblu, acest prim scenariu confirmă că Favigon poate susține un traseu complet chiar și fără AI: creare, editare, salvare, preview și export. Asta este relevant deoarece demonstrează că aplicația nu depinde complet de componenta generativă pentru a avea sens ca produs. AI-ul extinde sistemul, dar nu îl înlocuiește.

## **7.4 Scenariul 2: generare pornind de la prompt și rafinare ulterioară**

Al doilea scenariu urmărește zona care diferențiază cel mai mult proiectul: utilizarea AI-ului ca punct de pornire pentru construcția unei interfețe, urmată de integrarea rezultatului în editor și de rafinarea lui manuală. Acest scenariu este important pentru că arată dacă relația dintre prompt, pipeline, reprezentare intermediară și editare directă rămâne coerentă și în practică, nu doar la nivel conceptual.

Punctul de pornire este un prompt de nivel produs, de exemplu o cerere pentru o pagină landing a unei aplicații SaaS, cu secțiuni standard, accent pe call-to-action și stil vizual modern. În această etapă, utilizatorul nu descrie fiecare pixel și nici nu dictează structura exactă a arborelui intern. Rolul promptului este să exprime intenția, iar sistemul trebuie să o transforme într-o bază de lucru utilizabilă.

Din punct de vedere arhitectural, acesta este exact contextul în care devine vizibil avantajul pipeline-ului în trei faze. Dacă generarea ar produce direct cod final, orice neclaritate din rezultat ar fi greu de localizat. În schimb, în Favigon, drumul trece prin intenție, structură și stilizare. Asta face ca rezultatul final să poată fi discutat și corectat pe etape. Chiar dacă utilizatorul final nu vede explicit fiecare structură intermediară, efectul este unul important: sistemul oferă un rezultat mai controlabil.

După finalizarea pipeline-ului, rezultatul nu rămâne izolat într-o fereastră de răspuns și nici nu este oferit doar ca export static. El este integrat în editor, unde poate fi tratat ca document de lucru normal. Din perspectiva produsului, acesta este probabil cel mai important pas al întregii aplicații. Dacă designul generat nu ar putea fi preluat în editor, Favigon ar rămâne mai aproape de un generator de mockup-uri decât de o platformă de design asistat.

Odată încărcat în editor, utilizatorul poate modifica structura, poate rescrie texte, poate ajusta dimensiuni, spațieri, culori și layout-uri, sau poate adăuga elemente noi. În acest fel, AI-ul funcționează ca accelerator al primei versiuni, nu ca autor final incontestabil al interfeței. Acest aspect este important și pentru tonul academic al lucrării, deoarece arată o abordare realistă: modelul propune o soluție inițială, iar utilizatorul rămâne în controlul rezultatului.

În practică, etapa de rafinare este cea care validează calitatea integrării dintre generare și editare. Dacă rezultatul AI ar intra în editor într-o formă greu de manipulat, cu elemente poziționate arbitrar sau cu proprietăți incoerente, întreaga idee de continuitate s-ar pierde. Observația importantă aici este că reprezentarea intermediară acționează tocmai ca stratul care face posibilă această tranziție. Designul generat poate fi tratat în același limbaj intern ca un design creat manual, ceea ce simplifică atât editarea, cât și exportul ulterior.

În plus, scenariul acesta evidențiază și una dintre limitările reale ale aplicației. Calitatea rezultatului inițial depinde încă de prompt și de disciplina regulilor folosite în mesajele de sistem. Uneori, prima variantă este foarte aproape de ce se dorește; alteori, ea cere ajustări mai vizibile. Totuși, tocmai faptul că rezultatul poate fi corectat în editor fără ruperea fluxului face ca această limitare să fie mai ușor de acceptat în practică.

După rafinare, utilizatorul poate relua pașii de preview și export, ca în scenariul anterior. Din această perspectivă, scenariul 2 nu este complet diferit de scenariul 1, ci îl extinde. Diferența

este că punctul de intrare nu mai este o pagină goală, ci o propunere generată. În rest, traseul rămâne același: document editabil, preview realist și artefacte de ieșire inspectabile.

Acesta este un rezultat important pentru lucrare, deoarece arată că AI-ul nu introduce un “tunel” paralel în aplicație. El alimentează același flux general de produs. Din punct de vedere al proiectării software, aceasta este una dintre cele mai solide justificări pentru separarea dintre generare, reprezentare intermediară și conversie.

## 7.5 Scenariul 3: publicare, explorare și reutilizare prin fork

Al treilea scenariu urmărește trecerea din zona individuală de lucru în zona socială a aplicației. Pentru multe proiecte academice, această parte ar putea părea secundară, însă în cazul Favigon ea are un rol important: arată că rezultatele create nu rămân blocate în spațiul privat al unui singur utilizator, ci pot circula, pot fi descoperite și pot deveni puncte de plecare pentru alte proiecte.

Scenariul pornește de la un proiect considerat suficient de matur pentru a fi făcut public. După publicare, el devine eligibil pentru afișare în zona ‘Explore’, unde utilizatorii pot vedea proiecte populare sau recomandate și pot descoperi alți autori. Figura 5.1 este relevantă tocmai pentru această etapă, deoarece arată că aplicația nu se oprește la o listă administrativă de proiecte, ci oferă un spațiu distinct de descoperire.

Din punct de vedere al produsului, publicarea schimbă statutul proiectului. El nu mai este doar un document persistat în contul unui autor, ci o resursă care poate primi vizualizări, aprecieri și bookmark-uri și care poate fi asociată mai clar cu identitatea creatorului său. Acest lucru contează deoarece mută aplicația din zona unui instrument de lucru local către zona unui ecosistem mic, dar coerent, de creare și reutilizare.

O funcție deosebit de relevantă în acest context este fork-ul. Dacă un utilizator găsește un proiect public interesant, el poate crea o copie proprie pornind de la acel punct. Din perspectivă tehnică, această operație nu înseamnă doar duplicarea unui document JSON, ci păstrarea unei relații între proiectul-sursă și cel derivat. Din perspectivă practică, rezultatul este foarte util: reutilizarea devine concretă și trasabilă.

Acest scenariu este valoros și pentru că arată întâlnirea dintre două zone care altfel ar putea rămâne separate: partea de producție și partea socială. În multe produse, zona de explorare este doar un catalog de exemple. În Favigon, ea are efect direct asupra procesului de creație, deoarece un proiect descoperit public poate fi transformat într-un punct de pornire pentru o nouă iterație de editare și export.

Mai mult, acest scenariu oferă și un argument bun pentru existența infrastructurii de profiluri, follow, like, star și proiecte publice. Dacă lucrarea s-ar opri doar la generarea AI și la export, aceste funcții ar părea poate suplimentare. Însă, puse într-un flux concret de publicare, explorare și fork, ele capătă o justificare clară la nivel de produs.

## 7.6 Observații asupra comportamentului sistemului

Scenariile anterioare permit câteva observații generale despre comportamentul aplicației în utilizare practică.

Prima observație este că separarea pe subsisteme nu rupe experiența utilizatorului. Deși intern există granițe clare între editor, pipeline AI, convertor, backend REST și componenta socială, în utilizare acestea apar ca părți ale aceluiași produs. Acesta este un semn bun pentru proiect, fiindcă o arhitectură curată, dar resimțită de utilizator ca fragmentare, nu ar reprezenta un rezultat reușit.

A doua observație este că reprezentarea intermediară nu este doar o decizie tehnică elegantă, ci un element cu efect vizibil în flux. Ea face posibilă trecerea dintre editare, generare și export fără schimbarea “limbajului intern” al documentului. Din acest motiv, multe dintre operațiile discutate în scenariile anterioare par mai naturale decât ar părea într-un sistem bazat doar pe transformări ad-hoc între formate incompatibile.

A treia observație este că preview-ul joacă un rol mai important decât ar putea părea inițial. El nu este doar o etapă de prezentare, ci și un punct de control între documentul editabil și ieșirea finală. În practică, el ajută la separarea problemelor vizuale de problemele de conversie și reduce riscul ca utilizatorul să descopere abia la export că rezultatul nu corespunde așteptărilor.

A patra observație privește limitările rămase. Deși fluxul general este coerent, sistemul nu elimină complet costul rafinării manuale, mai ales în scenariile pornite din prompt. Pentru proiectul de față, însă, acest lucru nu trebuie privit ca eșec. O platformă academică realistă nu trebuie să pretindă că AI-ul înlocuiește complet editarea, ci că reduce costul pornirii și al iterării. Din această perspectivă, comportamentul actual este acceptabil și explicabil.

În fine, scenariile practice confirmă că Favigon nu este doar un demo izolat pentru una dintre componente. Editorul are utilitate și fără AI, AI-ul este util pentru accelerarea construcției, convertorul produce artefacte inspectabile, iar zona socială oferă un sens suplimentar proiectelor create. Împreună, acestea susțin ideea că aplicația funcționează ca produs unitar, chiar dacă există încă limite și zone extensibile.

## 7.7 Concluzii intermediare

Capitolul de față a urmărit aplicația dintr-o perspectivă pragmatică, orientată pe trasee reale de utilizare. Rezultatul principal este că fluxurile importante ale produsului pot fi parcurse într-o succesiune coerentă: creare, editare, generare asistată, preview, export, publicare și reutilizare. Acest lucru întărește observația că valoarea proiectului nu stă într-o singură funcționalitate spectaculoasă, ci în integrarea consistentă a mai multor subsisteme.

În capitolul următor sunt discutate măsurile de securitate, testarea automată și validarea sistemului, adică partea care completează observațiile practice de aici cu o analiză explicită a fiabilității și a limitelor de verificare.

## Capitolul 8

# Securitate, testare și validare

### 8.1 Rolul acestei etape în proiect

Într-un proiect de tip Favigon, securitatea și validarea nu pot fi tratate ca pași de final adăugați formal. Aplicația gestionează autentificare, sesiuni, proiecte private, upload de fișiere și integrare cu servicii externe costisitoare. În același timp, editorul și pipeline-ul AI au suficientă complexitate încât să necesite un efort real de verificare.

Din acest motiv, validarea a fost privită pe trei niveluri:

- **măsuri preventive în implementare**, care reduc suprafața de risc;
- **testare automată**, concentrată în special pe backend și convertor;
- **validare funcțională**, prin scenarii operaționale și fluxuri reale în interfață.

### 8.2 Măsuri de securitate implementate

#### 8.2.1 Autentificare și sesiune

Securitatea începe din zona de autentificare. În Favigon, autentificarea se bazează pe JWT bearer validat în backend, dar transportat către client prin cookie HTTP only. Refresh token-ul este separat și limitat la un path specific. Această alegere reduce expunerea directă a token-urilor în codul client și simplifică rotația lor la refresh [7, 25].

Pentru scenariile de risc mai mare, proiectul include și autentificare în doi pași. Această măsură crește costul unui atac bazat exclusiv pe compromiterea parolei [26]. Chiar dacă implementarea actuală folosește cod prin email și nu un factor hardware, ea rămâne o îmbunătățire reală față de autentificarea cu un singur factor.

#### 8.2.2 Limitarea expunerii API-ului

O altă măsură importantă este rate limiting-ul configurat diferențiat pe categorii de endpoint-uri. Zona de autentificare are praguri mai stricte, zona AI este limitată pentru a controla costul și abuzul, iar anumite endpoint-uri de utilizatori au propriile reguli. Documentația ASP.NET Core recomandă această abordare pentru protejarea aplicațiilor expuse public [18]. În configurația curentă, politicile observabile în 'Program.cs' sunt: 'auth' 10 cereri / minut, 'ai' 10 cereri / minut, 'converter' 30 cereri / minut cu coadă de 2 cereri și 'users' 60 cereri / minut.

În plus, backend-ul impune limite pentru dimensiunea cererilor. Cererile JSON globale au limită de 1 MB, iar endpoint-urile pentru upload sau flush folosesc praguri dedicate de 5 MB, 10 MB și 15 MB, în funcție de tipul operației. Este o măsură simplă, dar utilă împotriva folosirii necontrolate a resurselor.

#### 8.2.3 Middleware și antete de securitate

În 'Program.cs', backend-ul include middleware personalizat pentru tratarea excepțiilor și middleware pentru antete de securitate. În mediul non-development sunt activate HSTS și redirectionarea HTTPS. CORS este configurat explicit pe baza unei liste de origini

permise. Aceste decizii nu fac aplicația invulnerabilă, dar indică o preocupare practică pentru comportamentul în producție [20, 21].

## 8.2.4 Upload-uri și asset-uri

Proiectul permite încărcarea de imagini pentru profil și pentru proiecte. Din acest motiv, fișierele primite nu sunt tratate ca date “neutre”. Backend-ul validează tipul, dimensiunea și contextul în care upload-ul este permis. În plus, la actualizarea designului sunt identificate și șterse asset-urile orfane, tocmai pentru a evita acumularea de fișiere inutile.

**Tabela 8.1:** Măsură de securitate implementate în Favigon

| Zonă              | Măsură implementată                             | Rol practic                                                     |
|-------------------|-------------------------------------------------|-----------------------------------------------------------------|
| Autentificare     | JWT în cookie HTTP only + refresh token separat | reduce expunerea token-urilor și permite reînnoirea sesiunii    |
| 2FA               | cod temporar prin email                         | ridică nivelul de protecție la login și operații sensibile      |
| API public        | rate limiting pe politici distincte             | reduce abuzul și protejează resursele costisitoare              |
| Transport         | HSTS și HTTPS în producție                      | reduce riscul de trafic neprotejat                              |
| Browser/API       | CORS configurat explicit                        | limitează originile care pot face cereri autentificate          |
| Upload-uri        | validare de dimensiune și tip                   | reduce riscul de folosire abuzivă a endpoint-urilor de fișiere  |
| Tratarea erorilor | middleware dedicat pentru excepții              | uniformizează răspunsurile și evită scurgeri inutile de detalii |

Tabelul 8.1 rezumă măsurile principale, însă valoarea lor practică apare tocmai din combinația lor: autentificare bazată pe cookie HTTP only, rotație separată prin refresh token, politici de limitare a cererilor și restricții explicite pentru upload-uri.

## 8.3 Strategia de testare

### 8.3.1 Testare automată backend

La data de 1 iunie 2026, rularea locală a comenzii ‘dotnet test’ pentru proiectul ‘Backend/Favigon.Tests’ a raportat 108 teste trecute, 0 eșuate și o durată totală de aproximativ 6 secunde. Testele acoperă servicii, controllere, middleware și convertorul. Acest fapt este important pentru că zonele cele mai sensibile tehnic ale aplicației se află tocmai în backend și în motorul de conversie.

Mai precis, există teste pentru:

- autentificare: login, register, refresh, resetare parolă, 2FA;
- proiecte: controller și service;
- utilizatori: controller și service;
- convertor: controller și motorul propriu-zis;
- middleware: tratarea excepțiilor și antetele de securitate.

**Tabela 8.2:** Distribuția testelor automate backend

| <b>Zonă testată</b>          | <b>Ce verifică în principal</b>                           | <b>Nr. teste</b> |
|------------------------------|-----------------------------------------------------------|------------------|
| Autentificare și cont        | login, register, refresh, parole, 2FA, recuperare cont    | 25               |
| Servicii pentru proiecte     | gestionare proiecte, fork, bookmark, like și upload logic | 20               |
| Endpoint-uri pentru proiecte | comportamentul controllerului pentru proiecte             | 11               |
| Utilizatori și profil        | profil, follow, search, stele și operații auxiliare       | 17               |
| Convertor și export          | validare IR, export, responsive și multi-page             | 19               |
| Middleware și securitate     | tratarea erorilor și antetele HTTP                        | 16               |
| <b>Total</b>                 | <b>teste backend automate trecute</b>                     | <b>108</b>       |

### 8.3.2 Testarea convertorului

Convertorul merită menționat separat deoarece este una dintre zonele cele mai sensibile la regresii. Testele existente nu se opresc la cazul simplu de export pentru un singur ecran. Ele includ și cazuri pentru:

- normalizarea rădăcinii provenite din canvas;
- generarea de clase CSS distincte pentru nume duplicate;
- omiterea unor stiluri implicite nerelevante;
- propagarea modului de overflow;
- noduri exclusive pentru anumite breakpoint-uri;
- reordonarea copiilor în layout-uri flex;
- diferențe de pseudo-clase și keyframes în export responsive.

Această suită de teste este importantă deoarece confirmă faptul că motorul nu este doar un generator static. El tratează situații reale care apar atunci când aceeași pagină trebuie exportată în mai multe variante sau cu structuri ușor diferite pe breakpoint-uri diferite.

### 8.3.3 Scenarii reprezentative ancorate în teste

Dincolo de numărul total al testelor, este utilă și observarea unor scenarii concrete care există deja în suită. Acestea arată mai clar ce comportamente sensibile sunt verificate efectiv, nu doar enumerate generic. Exemplele sintetice sunt rezumate în Tabelul 8.3.

**Tabela 8.3:** Scenarii reprezentative verificate prin teste automate

| Zonă                | Scenariu testat                             | Comportament verificat                                                                                              |
|---------------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Autentificare 2FA   | login cu 2FA activ                          | este emis un token intermediar, se trimite codul pe email și sesiunea finală nu este acordată înainte de verificare |
| Autentificare 2FA   | verificarea codului valid de login          | token-urile finale sunt emise, iar starea temporară de challenge este ștearsă                                       |
| Persistență proiect | înlocuirea unui asset imagine               | asset-ul vechi nefolosit este eliminat, evitând acumularea de fișiere orfane                                        |
| Export multi-page   | normalizarea rădăcinii provenite din canvas | coordonatele specifice editorului nu sunt propagate incorect în CSS-ul paginii exportate                            |
| Export responsive   | reordonarea copiilor flex pe breakpoint     | diferențele de ordine sunt emise în media queries, fără duplicarea inutilă a întregii structuri                     |

### 8.3.4 Indicatori cantitativi observați

Pe lângă distribuția testelor din Tabelul 8.2, validarea curentă oferă și câteva valori concrete, extrase direct din rulările și configurațiile existente ale proiectului.

**Tabela 8.4:** Indicatori cantitativi observați în validarea curentă

| Indicator                     | Valoare observată                                                                                | Sursă                                                         |
|-------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Execuția suitei backend       | 108 / 108 teste trecute, 0 eșuate, aproximativ 6 s                                               | rulare 'dotnet test' în 'Backend/Favigon.Tests' la 01.06.2026 |
| Suprafața API expusă          | 6 controllere și 62 endpoint-uri anotate HTTP                                                    | inventar din controllerele API                                |
| Politici de rate limiting     | 4 politici distincte ('auth', 'ai', 'converter', 'users')                                        | configurarea din 'Program.cs'                                 |
| Persistare automată în editor | debounce de 500 ms, flush maxim 4 s, 4 reîncercări cu bază 2 s                                   | 'CanvasPersistenceService'                                    |
| Orchestrare AI                | cache TTL 10 minute, 3 faze, suport pentru oprire după faza 1 sau 2 și evenimente SSE de progres | 'AiDesignService' și 'AiPipelineService'                      |
| Limite pentru cereri          | 1 MB global pentru JSON și 5 / 10 / 15 MB pe endpoint-uri sensibile                              | 'Program.cs' și attribute 'RequestSizeLimit'                  |



### 8.3.5 Testare frontend

Pe partea de frontend, testarea automată este în acest moment mult mai redusă. În repository există doar un test minimal la nivelul aplicației, nu o suită comparabilă cu cea din backend. Este importantă precizarea explicită a acestui fapt, pentru că altfel s-ar crea impresia falsă că proiectul este testat uniform în toate straturile.

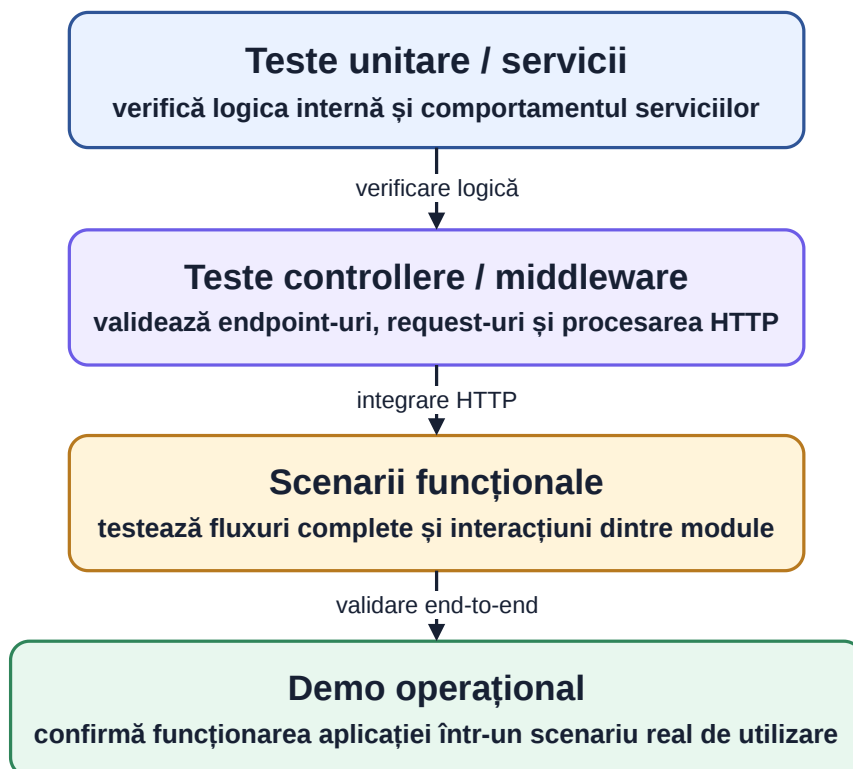
Motivul principal este complexitatea mare a editorului canvas și timpul limitat al proiectului. În locul unei suite largi, dar superficiale, efortul a fost direcționat spre stabilizarea logicii și spre validare funcțională directă. Nu este o soluție ideală pe termen lung, dar pentru stadiul actual al proiectului reprezintă un compromis explicit.

## 8.4 Validare funcțională

În completarea testelor automate, aplicația a fost verificată prin scenarii funcționale care urmăresc fluxurile principale ale produsului. La nivel de sinteză, ele acoperă:

1. creare cont, login și logout;
2. activare 2FA și autentificare cu pas suplimentar;
3. creare proiect nou și salvare automată;
4. editare în canvas cu mutare, redimensionare, selecție și text;
5. generare AI și integrarea rezultatului în editor;
6. export în HTML, React și Angular;
7. publicare, vizualizare publică, like, star și fork.

Această validare a fost utilă mai ales fiindcă a legat împreună subsisteme care, individual, puteau părea corecte. De exemplu, un export poate funcționa separat, dar dacă designul salvat nu este mapat corect în IR, fluxul cap-coadă devine fragil. La fel, 2FA poate funcționa la nivel de API, dar dacă interfața nu tratează bine starea intermediară, experiența utilizatorului se degradează rapid. Straturile complementare folosite pentru această verificare sunt reprezentate în Figura 8.1.



**Figura 8.1:** Straturi complementare de validare folosite în proiect

## 8.5 Limitări ale validării actuale

Deși proiectul are o bază bună de verificare, există și limite care trebuie menționate explicit.

Prima limită este testarea frontend-ului. Editorul canvas ar beneficia de teste automate mai bogate, în special pentru interacțiuni complexe și regresii vizuale.

A doua limită este partea de AI. Chiar dacă rezultatele pot fi validate structural, evaluarea calității estetice și a adecvării conținutului rămâne într-o anumită măsură dependentă de observația umană.

A treia limită este absența unor măsurători cantitative mai avansate pentru performanță sub sarcină. Proiectul are rate limiting și structură tehnică bună, dar nu a fost supus încă unei campanii complete de load testing.

## 8.6 Concluzii intermediare

Securitatea și validarea din Favigon nu rezolvă toate problemele posibile, dar oferă o bază credibilă pentru funcționarea produsului. Autentificarea, 2FA, limitarea cererilor, validarea upload-urilor și middleware-ul de protecție reduc riscuri concrete. În paralel, cele 108 teste backend și scenariile funcționale confirmă că aplicația nu este doar conceptual coerentă, ci și suficient de stabilă pentru demonstrare și utilizare controlată.

Pe baza acestei evaluări, capitolul final sintetizează rezultatele proiectului și direcțiile de dezvoltare viitoare.

## Capitolul 9

# Concluzii și perspective de dezvoltare

### 9.1 Gradul de îndeplinire a obiectivelor

Analiza din capitolele anterioare arată că obiectivul principal al lucrării a fost atins. A fost proiectată și implementată o platformă web care permite editare vizuală, generare asistată de AI, conversie în cod și reutilizarea proiectelor într-un context social. Mai important decât lista de funcții este faptul că acestea nu au rămas separate, ci au fost integrate într-un produs unitar.

Obiectivele specifice formulate la început au fost acoperite în mod satisfăcător. Arhitectura aplicației este separată clar pe frontend, API, aplicație, infrastructură și convertor. Editorul canvas suportă lucru multi-page, istoric, clipboard, gesturi și persistare automată. Reprezentarea intermediară leagă editorul, AI-ul și exportul. Pipeline-ul AI în trei faze funcționează controlat, iar exportul este disponibil pentru HTML, React și Angular. În plus, partea de produs include autentificare, proiecte publice și private, precum și mecanisme de tip like, star, follow și fork.

Pentru o lucrare de licență, este relevant și faptul că proiectul nu a rămas la nivelul unui demo local. Existența autentificării, a profilurilor, a testelor automate backend și a unei infrastructuri de rulare complete arată că soluția a fost tratată ca aplicație software coerentă, nu doar ca experiment punctual.

### 9.2 Puncte forte ale proiectului

Primul punct forte este coerența dintre subsisteme. Editorul, AI-ul, exportul și zona socială folosesc aceeași bază de proiect și aceeași infrastructură generală. În mod normal, aceste zone apar în produse diferite sau sunt tratate superficial atunci când sunt puse împreună. În Favigon, tocmai integrarea lor reprezintă partea cea mai valoroasă.

Al doilea punct forte este reprezentarea intermediară a interfeței. Ea a permis separarea dintre editare, generare și conversie, fără a lega platforma de un singur framework sau de o singură formă de ieșire. Din punct de vedere tehnic, aceasta este una dintre deciziile care au făcut proiectul extensibil.

Al treilea punct forte este modul în care a fost introdus AI-ul. În locul unei abordări de tip „prompt și cod final”, proiectul folosește etape, constrângeri și validări. Rezultatul nu elimină variabilitatea modelului, dar o face mai ușor de controlat și de explicat.

Un al patrulea punct forte este validarea backend-ului și a convertorului. Cele 108 teste automate nu acoperă tot produsul, dar oferă încredere în zonele cele mai sensibile: autentificare, proiecte, middleware și conversie.

### 9.3 Limitări

Ca orice proiect de licență, Favigon are și limite clare.

Prima limită este testarea frontend-ului, în special a editorului canvas. Deși comportamentele principale au fost verificate funcțional, acoperirea automată este încă redusă în comparație cu partea de backend.

A doua limită este dependența de calitatea ieșirilor AI. Chiar dacă pipeline-ul și validarea reduc din variabilitate, rezultatul final depinde în continuare de model și de promptul primit.

A treia limită ține de colaborare. Platforma permite publicare, fork și reutilizare, dar nu oferă încă editare simultană sau comentarii direct în interiorul proiectului.

A patra limită este nivelul de rafinare al exportului. Codul generat este util și coerent, dar poate fi îmbunătățit în viitor prin reguli mai avansate pentru fiecare ecosistem țintă și prin integrare mai bună cu design systems externe.

## 9.4 Perspective de dezvoltare

O direcție firească de continuare este introducerea colaborării în timp real și a unei forme mai bogate de versionare. Pentru un produs orientat și spre reutilizare, această extindere ar aduce mai multă valoare decât simpla adăugare de noi tipuri de export.

A doua direcție privește partea de AI. Pipeline-ul actual poate fi rafinat prin exemple contextuale, evaluare mai bună a rezultatului intermediar și transformări iterative pornind de la un design deja existent, nu doar de la un prompt nou.

A treia direcție este componentizarea la nivel de editor și de export. În forma actuală, proiectul lucrează bine cu pagini și secțiuni, dar un nivel mai clar de componente reutilizabile ar face platforma mai potrivită pentru aplicații mai mari.

A patra direcție importantă este extinderea testării și a observabilității. Pentru frontend ar fi utile teste end-to-end și verificări vizuale automate, iar pentru backend și AI ar ajuta metrice și loguri mai detaliate.

## 9.5 Încheiere

Favigon nu rezolvă complet toate problemele din zona design-to-code, dar demonstrează că un astfel de flux poate fi construit într-o formă coerentă și controlabilă. Editorul vizual, reprezentarea intermediară, pipeline-ul AI și motorul de conversie funcționează împreună suficient de bine încât să susțină un produs real, nu doar o demonstrație izolată.

Din punctul de vedere al lucrării, rezultatul este relevant tocmai pentru că îmbină decizii de arhitectură, experiență de utilizator, securitate, persistare și integrare AI în aceeași aplicație. În această formă, proiectul reprezintă atât un rezultat valid pentru lucrarea de licență, cât și o bază bună pentru dezvoltări ulterioare.

## Anexa A

# Exemple tehnice suplimentare

### A.1 Exemplu simplificat de reprezentare intermediară

În listarea următoare prezint un exemplu simplificat de arbore IR folosit pentru descrierea unei secțiuni de tip hero. În implementarea reală, structura poate include mai multe câmpuri și mai multe variante responsive, însă ideea de bază rămâne aceeași: fiecare nod descrie tipul, dimensiunea, layout-ul și conținutul relevant.

**Listing A.1:** Exemplu simplificat de IR pentru o secțiune hero

```
{
  "id": "frame-home",
  "type": "Frame",
  "meta": { "name": "Home Page" },
  "style": {
    "width": { "value": 1280, "unit": "px" },
    "height": { "value": 0, "unit": "fit-content" }
  },
  "children": [
    {
      "id": "hero-section",
      "type": "Container",
      "meta": { "name": "Hero Section" },
      "layout": {
        "mode": "Flex",
        "direction": "Row",
        "gap": { "value": 48, "unit": "px" }
      },
      "style": {
        "padding": {
          "top": { "value": 96, "unit": "px" },
          "right": { "value": 80, "unit": "px" },
          "bottom": { "value": 96, "unit": "px" },
          "left": { "value": 80, "unit": "px" }
        }
      },
      "children": [
        {
          "id": "hero-title",
          "type": "Text",
          "props": { "text": "șConstruiete ținterfee și ăexport-le în cod" }
        }
      ]
    }
  ]
}
```

Acest exemplu arată de ce IR-ul este util. El nu conține numai text sau numai stil, ci și relațiile de ierarhie și layout. Astfel, același document poate fi randat într-un canvas, validat și apoi convertit într-o implementare concretă.

## A.2 Exemple simplificate de cereri către API

În această anexă este utilă și o vedere foarte concretă asupra formei cererilor transmise către backend pentru funcțiile cele mai relevante tehnic. Exemplele de mai jos nu reproduc toate câmpurile posibile din implementarea reală, dar păstrează structura esențială a payload-urilor.

**Listing A.2:** Exemplu simplificat de request pentru pipeline-ul AI

```
{
  "prompt": "Landing page pentru o aplicatie SaaS de task management",
  "viewportWidth": 1280,
  "model": "gpt-5.4-mini",
  "stopAfterPhase": 3,
  "existingIr": null
}
```

**Listing A.3:** Exemplu simplificat de request pentru generarea de cod

```
{
  "framework": "html",
  "ir": {
    "id": "frame-home",
    "type": "Frame",
    "children": []
  }
}
```

În practică, aceste două exemple ilustrează două tipuri foarte diferite de cereri. Prima cere procesare probabilistică, etapizată și validată semantic. A doua cere transformare deterministă dintr-o structură deja formalizată. Tocmai această separare este una dintre ideile arhitecturale importante ale proiectului.

## A.3 Exemplu simplificat de cod generat

**Listing A.4:** Exemplu scurt de HTML generat de convertor

```
<section class="hero-section">
  <div class="hero-copy">
    <h1 class="hero-title">§Construiete ținterfee și ăexport-le în cod</h1>
    <p class="hero-body">§Pornete de la un prompt, ărafineaz în editor și ăexport rezultatul.</p>
  </div>
</section>
```

**Listing A.5:** Exemplu scurt de CSS generat de convertor

```
.hero-section {
  display: flex;
  flex-direction: row;
  gap: 48px;
  padding: 96px 80px;
}

.hero-title {
  font-size: 64px;
  font-weight: 800;
}
```

---

## A.4 Exemplu simplificat de evenimente de streaming

În scenariile de generare incrementală, frontend-ul nu primește doar un răspuns final, ci o secvență de evenimente SSE. Varianta simplificată de mai jos redă forma protocolului folosit pentru notificarea progresului și pentru transmiterea rezultatului final.

**Listing A.6:** Exemplu simplificat de flux SSE pentru pipeline-ul AI

```
event: phase_start
data: {"phase":1,"name":"intent"}

event: phase_complete
data: {"phase":1,"name":"intent"}

event: phase_start
data: {"phase":2,"name":"structure"}

event: result
data: {"success":true,"stopAfterPhase":3}
```

Din punct de vedere al interfeței, acest format face posibilă afișarea unui progres real, nu doar a unei stări generice de așteptare. Din punct de vedere al depanării, el ajută la observarea etapei în care apare o eventuală eroare sau oprire prematură.

## Anexa B

# Inventar de endpoint-uri API

## B.1 Observații generale

Această anexă sintetizează endpoint-urile principale ale aplicației. Nu urmăresc să refac documentația Swagger în întregime, ci să ofer o imagine clară asupra suprafeței API-ului și a modului în care aceasta este împărțită pe domenii funcționale.

## B.2 Endpoint-uri pentru cont și autentificare

**Tabela B.1:** Endpoint-uri principale din zona de autentificare

| Metodă | Rută                                    | Rol                                       |
|--------|-----------------------------------------|-------------------------------------------|
| POST   | /api/account/register                   | creare cont nou                           |
| POST   | /api/account/login                      | autentificare clasică                     |
| POST   | /api/account/oauth2/github              | autentificare prin GitHub                 |
| POST   | /api/account/oauth2/google              | autentificare prin Google                 |
| POST   | /api/account/two-factor/verify-login    | finalizarea login-ului cu 2FA             |
| POST   | /api/account/oauth2/github/link         | legare cont GitHub la utilizatorul curent |
| POST   | /api/account/oauth2/google/link         | legare cont Google la utilizatorul curent |
| POST   | /api/account/forgot-password            | inițiere resetare parolă                  |
| POST   | /api/account/reset-password             | setarea noii parole prin token            |
| POST   | /api/account/set-password               | adăugarea unei parole pentru cont OAuth   |
| POST   | /api/account/change-password            | schimbarea parolei                        |
| POST   | /api/account/two-factor/enable/request  | trimiterea codului pentru activare 2FA    |
| POST   | /api/account/two-factor/enable/confirm  | confirmarea activării 2FA                 |
| POST   | /api/account/two-factor/disable/request | trimiterea codului pentru dezactivare 2FA |
| POST   | /api/account/two-factor/disable/confirm | confirmarea dezactivării 2FA              |
| POST   | /api/account/refresh                    | reînnoirea token-ului de acces            |
| POST   | /api/account/logout                     | încheierea sesiunii                       |

## B.3 Endpoint-uri pentru proiecte



**Tabela B.2:** Endpoint-uri principale pentru proiecte

| Metodă | Rută                             | Rol                                     |
|--------|----------------------------------|-----------------------------------------|
| GET    | /api/projects                    | lista proiectelor utilizatorului curent |
| GET    | /api/projects/{id}               | proiect după identificator              |
| GET    | /api/projects/by-slug/{slug}     | proiect după slug                       |
| POST   | /api/projects                    | creare proiect                          |
| PUT    | /api/projects/{id}               | actualizare metadate proiect            |
| DELETE | /api/projects/{id}               | ștergere proiect                        |
| POST   | /api/projects/{id}/assets/images | upload imagine pentru canvas            |
| GET    | /api/projects/{id}/design        | încărcare design salvat                 |
| PUT    | /api/projects/{id}/design        | salvare design                          |
| POST   | /api/projects/{id}/flush         | salvare combinată design + thumbnail    |
| PUT    | /api/projects/{id}/thumbnail     | actualizare thumbnail                   |
| GET    | /api/projects/user/{userId}      | proiecte publice ale unui utilizator    |
| POST   | /api/projects/{id}/star          | bookmark / star                         |
| DELETE | /api/projects/{id}/star          | eliminare bookmark                      |
| POST   | /api/projects/{id}/view          | înregistrare vizualizare                |
| POST   | /api/projects/{id}/like          | apreciere proiect                       |
| DELETE | /api/projects/{id}/like          | eliminare apreciere                     |
| POST   | /api/projects/{id}/fork          | creare fork după proiect public         |

## B.4 Endpoint-uri pentru utilizatori și profiluri

**Tabela B.3:** Endpoint-uri principale pentru utilizatori

| Metodă | Rută                                     | Rol                                       |
|--------|------------------------------------------|-------------------------------------------|
| GET    | /api/users/me                            | profilul utilizatorului curent            |
| PUT    | /api/users/me                            | actualizare profil propriu                |
| POST   | /api/users/me/profile-image              | upload imagine de profil                  |
| DELETE | /api/users/me                            | ștergere cont propriu                     |
| DELETE | /api/users/me/linked-accounts/{provider} | deconectare provider OAuth                |
| GET    | /api/users/search?q=...                  | căutare de utilizatori                    |
| GET    | /api/users/{username}                    | profil public după username               |
| POST   | /api/users/{username}/follow             | urmărire utilizator                       |
| DELETE | /api/users/{username}/follow             | oprire urmărire                           |
| GET    | /api/users/{username}/followers          | lista urmăritorilor                       |
| GET    | /api/users/{username}/following          | lista utilizatorilor urmăriți             |
| GET    | /api/users/me/stars                      | proiectele salvate de utilizatorul curent |

## B.5 Endpoint-uri pentru explore și recomandări

Tabela B.4: Endpoint-uri din zona explore

| Metodă | Rută                     | Rol                                                |
|--------|--------------------------|----------------------------------------------------|
| GET    | /api/explore/trending    | proiecte populare                                  |
| GET    | /api/explore/recommended | recomandări de proiecte,<br>eventual personalizate |
| GET    | /api/explore/people      | sugestii de utilizatori                            |

## B.6 Endpoint-uri pentru AI și conversie

Tabela B.5: Endpoint-uri pentru AI și converter

| Metodă | Rută                           | Rol                                       |
|--------|--------------------------------|-------------------------------------------|
| POST   | /api/ai/design                 | generare design într-o singură cerere     |
| POST   | /api/ai/design/stream          | generare design în regim SSE              |
| POST   | /api/ai/design/pipeline        | runare pipeline AI în trei faze           |
| POST   | /api/ai/design/pipeline/stream | runare pipeline AI cu streaming           |
| POST   | /api/converter/generate        | export pentru o pagină sau set responsive |
| POST   | /api/converter/validate        | validare structură IR                     |
| POST   | /api/converter/generate-files  | generare fișiere pentru export multi-page |

### Observație finală.

Din inventarul de mai sus se vede că API-ul nu este limitat la o singură zonă funcțională. El trebuie să deservească atât fluxuri de produs clasice, cât și funcționalități tehnice speciale, cum sunt streaming-ul AI și exportul multi-framework. Tocmai această diversitate justifică structura pe straturi din backend și separarea convertorului în proiect propriu.

# Bibliografie

- [1] Angular Team, *Angular Routing*, 2026, URL: <https://angular.dev/guide/routing> (accesat în 23.5.2026).
- [2] Angular Team, *provideRouter*, 2026, URL: <https://angular.dev/api/router/provideRouter> (accesat în 23.5.2026).
- [3] Angular Team, *Signals — Overview*, 2026, URL: <https://angular.dev/guide/signals> (accesat în 23.5.2026).
- [4] Angular Team, *Standalone Components*, 2026, URL: <https://angular.dev/reference/migrations/standalone> (accesat în 23.5.2026).
- [5] Len Bass, Paul Clements și Rick Kazman, *Software Architecture in Practice*, a 4-a ed., Addison-Wesley, 2021.
- [6] Tony Beltramelli, „pix2code: Generating Code from a Graphical User Interface Screenshot”, în *arXiv preprint arXiv:1705.07962* (2017), URL: <https://arxiv.org/abs/1705.07962> (accesat în 23.5.2026).
- [7] Damien Bowden, *Configure JWT bearer authentication in ASP.NET Core*, 2025, URL: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/configure-jwt-bearer-authentication?view=aspnetcore-9.0> (accesat în 23.5.2026).
- [8] Builder.io, *Builder Visual Development Platform*, 2026, URL: <https://www.builder.io/visual-development-platform> (accesat în 23.5.2026).
- [9] Builder.io, *Code, create, iterate faster with AI*, 2026, URL: <https://www.builder.io/ai> (accesat în 23.5.2026).
- [10] Ravi Chugh et al., „Programmatic and Direct Manipulation, Together at Last”, în *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ACM, 2016, pp. 341–354, DOI: [10.1145/2908080.2908103](https://doi.org/10.1145/2908080.2908103), URL: <https://doi.org/10.1145/2908080.2908103> (accesat în 25.5.2026).
- [11] Cloudflare, *Cloudflare Tunnel*, 2026, URL: <https://developers.cloudflare.com/tunnel/> (accesat în 23.5.2026).
- [12] Docker, *Compose file reference*, 2026, URL: <https://docs.docker.com/compose/compose-file/> (accesat în 23.5.2026).
- [13] Docker, *Docker Compose*, 2026, URL: <https://docs.docker.com/compose/> (accesat în 23.5.2026).
- [14] Roy Thomas Fielding, „Architectural Styles and the Design of Network-based Software Architectures”, Teză de doct., University of California, Irvine, 2000, URL: [https://ics.uci.edu/~fielding/pubs/dissertation/fielding\\_dissertation.pdf](https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf) (accesat în 23.5.2026).
- [15] Figma, *Dev Mode: Design-to-Development*, 2026, URL: <https://www.figma.com/dev-mode/> (accesat în 23.5.2026).
- [16] Framer, *Wireframer*, 2026, URL: <https://www.framer.com/wireframer/> (accesat în 23.5.2026).
- [17] Ben Hutton et al., *JSON Schema Validation: A Vocabulary for Structural Validation of JSON*, 2022, URL: <https://json-schema.org/draft/2020-12/json-schema-validation> (accesat în 23.5.2026).
- [18] Arvin Kahbazi, Maarten Balliauw și Rick Anderson, *Rate limiting middleware in ASP.NET Core*, 2025, URL: <https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit?view=aspnetcore-10.0> (accesat în 23.5.2026).

- [19] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.
- [20] Microsoft Learn, *Enable Cross-Origin Requests (CORS) in ASP.NET Core*, 2024, URL: <https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-7.0> (accesat în 23.5.2026).
- [21] Microsoft Learn, *Handle errors in ASP.NET Core*, 2025, URL: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/error-handling?view=aspnetcore-9.0> (accesat în 23.5.2026).
- [22] Npgsql Development Team, *Npgsql Entity Framework Core Provider*, 2026, URL: <https://www.npgsql.org/efcore/?tabs=aspnet> (accesat în 23.5.2026).
- [23] OpenAI, *Chat Completions API Reference*, 2026, URL: <https://developers.openai.com/api/reference/resources/chat> (accesat în 23.5.2026).
- [24] OpenAI, *Structured model outputs*, 2026, URL: <https://platform.openai.com/docs/guides/structured-outputs?api-mode=chat> (accesat în 23.5.2026).
- [25] OWASP Foundation, *Authentication Cheat Sheet*, 2026, URL: [https://cheatsheetseries.owasp.org/cheatsheets/Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html) (accesat în 23.5.2026).
- [26] OWASP Foundation, *Multifactor Authentication Cheat Sheet*, 2026, URL: [https://cheatsheetseries.owasp.org/cheatsheets/Multifactor\\_Authentication\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html) (accesat în 23.5.2026).
- [27] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2026, URL: <https://www.postgresql.org/docs/16/> (accesat în 23.5.2026).
- [28] Yichi Shen et al., „UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback”, în *arXiv preprint arXiv:2406.07739* (2024), URL: <https://arxiv.org/abs/2406.07739> (accesat în 23.5.2026).
- [29] Ben Shneiderman, „Direct Manipulation: A Step Beyond Programming Languages”, în *Computer* 16.8 (1983), pp. 57–69, DOI: [10.1109 / MC . 1983 . 1654471](https://doi.ieeecomputersociety.org/10.1109/MC.1983.1654471), URL: <http://doi.ieeecomputersociety.org/10.1109/MC.1983.1654471> (accesat în 25.5.2026).
- [30] Yuxuan Wan et al., „Automatically Generating UI Code from Screenshot: A Divide-and-Conquer-Based Approach”, în *arXiv preprint arXiv:2406.16386* (2024), DOI: [10.48550/arXiv.2406.16386](https://arxiv.org/abs/2406.16386), URL: <https://arxiv.org/abs/2406.16386> (accesat în 1.6.2026).
- [31] Webflow, *Visual website builder*, 2026, URL: <https://webflow.com/> (accesat în 23.5.2026).
- [32] Austin Wright et al., *JSON Schema Draft 2020-12*, 2022, URL: <https://json-schema.org/draft/2020-12> (accesat în 23.5.2026).
- [33] Jinyu Xu et al., „LaTCoder: Converting Webpage Design to Code with Layout-as-Thought”, în *arXiv preprint arXiv:2508.03560* (2025), URL: <https://arxiv.org/abs/2508.03560> (accesat în 23.5.2026).